

Author
Andreas Oberdammer,
BSc
11908776

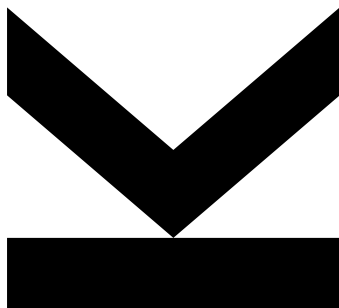
Submission
Institute of
Machine Learning

Thesis Supervisor
Univ. Prof. Mag. Dr.
Günther Klambauer

Assistant Thesis Supervisor
Dr. **Philipp Seidl**, MSc

December 2025

Evaluating xLSTM for EEG Seizure Classification



Master's Thesis

to confer the academic degree of

Master of Science

in the Master's Program

Artificial Intelligence

Abstract

This thesis evaluates the integration of Extended Long Short-Term Memory (xLSTM) architectures within the Biosignal Transformer (BIOT) framework for biosignal classification tasks. While BIOT’s original architecture employs Linear Transformers for processing tokenized biosignal sequences, this work investigates whether xLSTM variants can better capture temporal patterns in the data.

The study compares four model configurations on the Temple University EEG Event Corpus (TUEV) dataset: the original Linear Transformer-based BIOT, Matrix Long Short-Term Memory (mLSTM), Scalar Long Short-Term Memory (sLSTM), and combined mLSTM+sLSTM variants. Experiments were conducted across three temporal window lengths (5 s, 7 s, 9 s) with five random seeds per configuration, using standard evaluation metrics including Balanced Accuracy, Cohen’s Kappa, Weighted F1 Score, and AUCPR Macro. The experimental setup maintained consistency with the original BIOT evaluation protocol while incorporating architectural modifications to replace the Linear Transformer component with xLSTM blocks.

Results reveal window-dependent architectural advantages. sLSTM achieves the best overall performance of $52.46\% \pm 3.81\%$ balanced accuracy at 9-second windows, representing an 8.3% relative improvement over the baseline ($48.45\% \pm 3.73\%$). At 7 s windows, mLSTM performs best ($50.66\% \pm 2.87\%$), while sLSTM excels at 5 s ($52.15\% \pm 1.37\%$) with notably low variance. However, statistical significance testing reveals limited robust improvements: only AUCPR Macro at 5 s windows ($p = 0.005$) and overall accuracy at 9 s windows ($p = 0.029$) reach significance, with balanced accuracy differences remaining non-significant across all configurations. The combined architecture shows consistent stability but never achieves best-in-class performance.

The findings indicate that while xLSTM architectures provide window-specific numerical improvements, these benefits are modest, inconsistent across metrics, and come at substantial computational cost ($2.3\text{--}3.4\times$ training time overhead). The study highlights the importance of statistical validation in architectural comparisons and reveals practical trade-offs in deploying modern sequence models for clinical Electroencephalogram (EEG) classification. This work contributes empirical evidence about both the potential and the practical limitations of xLSTM architectures in biosignal processing applications, emphasizing that architectural advances do not automatically translate to clinically meaningful improvements in imbalanced classification tasks.

Kurzfassung

Diese Arbeit untersucht die Integration von Extended Long Short-Term Memory (xLSTM)-Architekturen in das Biosignal Transformer (BIOT)-Framework zur automatisierten Klassifikation von Biosignalen. Während das ursprüngliche BIOT-Modell auf Linear-Transformer-Blöcken basiert, wird hier geprüft, ob xLSTM-Varianten zeitliche Muster in den Daten effektiver erfassen können.

Die Evaluierung umfasst vier Modellvarianten – Linear Transformer, mLSTM, sLSTM sowie eine kombinierte mLSTM+sLSTM-Konfiguration – und erfolgt auf dem TUEV-Datensatz. Für jede Architektur wurden drei Fensterlängen (5 s, 7 s, 9 s) mit jeweils fünf Zufallsläufen getestet. Als Leistungskennzahlen dienten Balanced Accuracy, Cohen’s Kappa, gewichteter F1-Score und AUCPR (Macro). Das experimentelle Vorgehen orientierte sich eng am ursprünglichen BIOT-Protokoll, wobei ausschließlich die Sequenzverarbeitungsblöcke durch xLSTM-Strukturen ersetzt wurden.

Die Ergebnisse zeigen, dass die Leistungsfähigkeit stark von der gewählten Fensterlänge abhängt. sLSTM erzielt die besten Resultate bei 9-Sekunden-Fenstern ($52,46\% \pm 3,81\%$ Balanced Accuracy) und übertrifft die Baseline damit um rund 8 %. mLSTM erreicht seine beste Leistung bei 7 s ($50,66\% \pm 2,87\%$), während sLSTM bei 5 s ($52,15\% \pm 1,37\%$) besonders stabile Ergebnisse liefert. Statistisch signifikante Verbesserungen zeigen sich jedoch nur vereinzelt – für AUCPR bei 5 s ($p = 0,005$) und die Gesamtgenauigkeit bei 9 s ($p = 0,029$). Unterschiede in der Balanced Accuracy bleiben insgesamt nicht signifikant. Die kombinierte Architektur ist zwar sehr stabil, erreicht jedoch in keiner Konfiguration Spitzenwerte.

Insgesamt weisen die Resultate auf fensterabhängige, aber nur moderate Verbesserungen durch xLSTM-Modelle hin, die zudem mit deutlich höherem Rechenaufwand (2,3–3,4× längere Trainingszeit) verbunden sind. Die Studie betont die Bedeutung statistisch abgesicherter Vergleiche und zeigt, dass moderne Sequenzmodelle in der klinischen EEG-Analyse nur begrenzte praktische Vorteile bieten. Sie liefert damit empirische Evidenz für das Potenzial, aber auch die klaren Grenzen von xLSTM-Architekturen in der Biosignalverarbeitung und unterstreicht, dass architektonische Fortschritte nicht automatisch zu klinisch relevanten Verbesserungen bei unausgeglichene Klassifikationsaufgaben führen.

Contents

Abstract	ii
Kurzfassung	iii
List of Acronyms	vii
1 Introduction	1
1.1 Motivation	1
1.2 Objectives and Approach	2
1.2.1 Research Questions	2
1.2.2 Evaluation Methodology	2
1.2.3 Statistical Analysis and Visualization	2
1.2.4 Dataset and Scope	3
1.3 Outline	4
2 Background and Related Work	5
2.1 Biosignal Processing and BIOT Architecture	5
2.1.1 Challenges in Biosignal Processing	5
2.1.2 BIOT: Biosignal Transformer Architecture	5
2.2 Transformer Architecture and Linear Attention	10
2.2.1 Self-Attention Mechanism	10
2.2.2 Computational Complexity Challenges	11
2.2.3 Linear Attention in BIOT	11
2.2.4 Transformer in Biosignal Context	11
2.3 Extended Long Short-Term Memory (xLSTM)	12
2.3.1 Limitations of Traditional LSTMs	12
2.3.2 xLSTM Innovations	12
2.3.3 xLSTM Block Architecture	14
2.3.4 Theoretical Advantages for Biosignals	15
2.4 Integration of xLSTM with BIOT	15
2.4.1 Architectural Compatibility	15
2.4.2 Theoretical Synergies	16
2.4.3 Expected Benefits	16
3 Methodology	17
3.1 Experimental Framework	17
3.1.1 Research Questions and Objectives	17
3.1.2 Integration Strategy	17
3.1.3 Model Variants	18
3.2 Dataset: Temple University Hospital EEG Event Corpus	18
3.2.1 Background and Motivation	18
3.2.2 Dataset Structure and Organization	19
3.2.3 Event Classes and Clinical Significance	19
3.2.4 Class Distribution and Imbalance	20

3.2.5	Electrode Configuration and Channel System	20
3.3	Data Preprocessing Pipeline	21
3.3.1	Bipolar Montage Conversion	22
3.3.2	Signal Resampling	23
3.3.3	Signal Normalization	23
3.3.4	Event Window Extraction	23
3.4	Experimental Design	24
3.4.1	Training Strategy	24
3.4.2	Evaluation Methodology	24
3.4.3	Evaluation Metrics	25
3.4.4	Validation Protocol	26
3.5	Statistical Analysis Framework	26
3.5.1	Performance Aggregation	26
3.5.2	Statistical Significance Testing	27
3.5.3	Comparative Analysis	27
4	Experimental Setup	28
4.1	Implementation Environment	28
4.1.1	Software Stack	28
4.1.2	Hardware Configuration	29
4.2	Model Configuration	29
4.2.1	BIOT Tokenization Parameters	29
4.2.2	Baseline Transformer Configuration	30
4.2.3	xLSTM Component Configuration	31
4.2.4	Model Parameter Counts	32
4.2.5	Classification Head Configuration	32
4.3	Training Configuration	32
4.3.1	Optimization Parameters	32
4.3.2	Data Loading Configuration	33
4.3.3	Checkpointing Configuration	33
4.4	Experimental Execution	33
4.4.1	Parameter Sweep Implementation	33
5	Results and Analysis	35
5.0.1	Model Selection Criterion	35
5.1	Overall Performance Comparison	35
5.1.1	Complete Performance Results	35
5.1.2	Architectural Comparison (Across Windows)	36
5.2	Statistical Significance Testing	37
5.2.1	5-Second Windows	37
5.2.2	7-Second Windows	38
5.2.3	9-Second Windows	38
5.2.4	Interpretation	38
5.3	Computational Efficiency Analysis	39
5.4	Class-Specific Performance Analysis	39
5.4.1	Per-Class Recall Patterns	40
5.4.2	Statistical Comparison of Architectural Effects	42
5.5	Summary of Key Findings	44
5.5.1	Performance Characteristics	44
5.5.2	Statistical Significance	45
5.5.3	Architectural Insights	45

5.5.4	Practical Implications	45
6	Discussion	46
6.1	Performance Interpretation and Window Effects	46
6.1.1	Window-Dependent Benefits	46
6.1.2	Architecture-Specific Window Behaviors	47
6.1.3	Domain Transfer Challenges	47
6.2	Statistical Significance and Clinical Limitations	48
6.2.1	Statistical Power and Sample Size	48
6.2.2	Class Imbalance and Deployment Barriers	48
6.3	Study Limitations	48
6.3.1	Methodological Limitations	49
6.3.2	Architectural Limitations	49
6.3.3	Dataset and Evaluation Limitations	49
6.4	Summary	49
7	Conclusion and Outlook	51
7.1	Summary of Findings	51
7.2	Key Contributions	51
7.3	Limitations	52
7.4	Future Research Directions	52
7.4.1	Priorities	52
7.4.2	Methodological Advances	53
7.4.3	Long-Term Directions	53
7.5	Final Reflection	54
	Bibliography	55
	Appendix A Appendix	57
A.1	Research Questions and Answers	57
A.1.1	What is the central question?	57
A.1.2	Why is this question important?	57
A.1.3	What evidence/data (variables) are needed to answer this question?	58
A.1.4	What methods are used to get this evidence/data?	58
A.1.5	What analyses must be applied to the data to answer the central question?	59
A.1.6	What evidence/data (values for the variables) were obtained?	59
A.1.7	What were the results of the analyses?	60
A.1.8	How did the analyses answer the central question?	60
A.1.9	What does this answer tell us about the broader field?	61
	Appendix B Appendix	64
B.1	Additional Metric Figures	64
B.2	Complete Performance Metrics Per Run	64
B.2.1	Individual Run Performance Metrics by Architecture	64
B.2.2	Complete Experimental Runtime Data	69
B.3	Complete Confusion Matrix Collection	70
B.3.1	Overview	70
	List of Tables	74
	List of Figures	76

List of Acronyms

AI	Artificial Intelligence
AR	Average Reference
ARTF	Artifact
AUCPR	Area Under the Precision-Recall Curve
AUROC	Area Under the Receiver Operating Characteristic Curve
BCKG	Background
BIOT	Biosignal Transformer
CHB-MIT	Children’s Hospital Boston - Massachusetts Institute of Technology
Cls	Class
CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
CV	Coefficient of Variation
DDP	Distributed Data Parallel
DL	Deep Learning
ECG	Electrocardiogram
EDF	European Data Format
EEG	Electroencephalogram
ELU	Exponential Linear Unit
EMG	Electromyography
FFT	Fast Fourier Transform
EYEM	Eye Movement
GPED	Generalized Periodic Epileptiform Discharge
GPU	Graphics Processing Unit
ICU	Intensive Care Unit
LE	Linked Ears
LSTM	Long Short-Term Memory
ML	Machine Learning
mLSTM	Matrix Long Short-Term Memory
MGH	Massachusetts General Hospital

NFC	Near Field Communication
NLP	Natural Language Processing
PLED	Periodic Lateralized Epileptiform Discharge
RNN	Recurrent Neural Network
RTX	Ray Tracing Texel eXtreme
SHHS	Sleep Heart Health Study
sLSTM	Scalar Long Short-Term Memory
SPSW	Spike and Sharp Wave
STFT	Short-Time Fourier Transform
TCP	Temporal Central Parasagittal
TUAB	Temple University Abnormal EEG Corpus
TUEV	Temple University EEG Event Corpus
TUH	Temple University Hospital
xLSTM	Extended Long Short-Term Memory

Chapter 1

Introduction

1.1 Motivation

Biological signals, particularly Electroencephalogram (EEG), play a crucial role in numerous clinical applications, from neurological disease detection to sleep stage classification [20]. The development of deep learning models for biosignal analysis has demonstrated impressive success across diverse applications, including seizure detection, emotion analysis, and motor imagery recognition [20]. However, current deep learning models for biosignals are typically specialized for specific datasets and clinical settings, limiting their broader applicability.

Traditional approaches to biosignal modeling using Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), and Transformer architectures face significant challenges when dealing with real-world data variability. These challenges include mismatched channel configurations, variable sample lengths, and missing values that commonly arise from varying recording devices, system errors, and clinical limitations [20]. While existing models require extensive data preprocessing to align formats across datasets—often involving truncating, padding, or imputing data—such practices can introduce noise and shift data distributions, leading to poor generalization performance.

The Biosignal Transformer (BIOT) architecture addresses these limitations by introducing a novel biosignal tokenization approach that transforms diverse biosignals into unified "sentence" structures [20]. BIOT employs Linear Transformers to process these tokenized sequences, enabling cross-data learning and knowledge transfer across different biosignal datasets. However, recent advances in sequence modeling, particularly the development of Extended Long Short-Term Memory (xLSTM), present new opportunities for enhancing temporal dependency modeling in biosignals.

xLSTM architectures overcome traditional Long Short-Term Memory (LSTM) limitations [13] through exponential gating mechanisms and novel memory structures [2]. The xLSTM family includes Scalar Long Short-Term Memory (sLSTM) with scalar memory and new memory mixing capabilities, and Matrix Long Short-Term Memory (mLSTM) with matrix memory that is fully parallelizable [2]. These architectural improvements address key limitations of traditional LSTMs, including the inability to revise storage decisions, limited storage capacities, and lack of parallelizability.

Given the superior temporal modeling capabilities of xLSTM architectures and the successful application of BIOT in biosignal processing, investigating the integration of xLSTM components within the BIOT framework represents a promising research direction. This integration could potentially enhance the model's ability to capture complex temporal dependencies in biosignal data while maintaining the flexibility and cross-dataset applicability that makes BIOT effective.

1.2 Objectives and Approach

The primary objective of this thesis is to evaluate the effectiveness of integrating xLSTM architectures within the BIOT framework for biosignal classification tasks. Specifically, this work aims to determine whether xLSTM variants can enhance temporal dependency modeling compared to the original Linear Transformer-based BIOT architecture.

1.2.1 Research Questions

This thesis addresses the following key research questions:

1. Can xLSTM architectures effectively replace Linear Transformers in the BIOT framework while maintaining or improving EEG classification performance?
2. Which xLSTM variant (mLSTM, sLSTM, or combined configurations) demonstrates superior performance for EEG classification tasks?
3. How do different experimental configurations, particularly varying sample lengths, affect the relative performance of xLSTM variants compared to the baseline Linear Transformer?

1.2.2 Evaluation Methodology

To address these research questions, this thesis employs a comprehensive experimental approach using the TUEV dataset. The evaluation compares four distinct model configurations:

- **Baseline:** Original Linear Transformer-based BIOT architecture
- **mLSTM:** BIOT with Matrix LSTM component
- **sLSTM:** BIOT with Scalar LSTM component
- **M+S-LSTM:** BIOT with combined mLSTM and sLSTM components

The experimental design maintains consistency with the original BIOT evaluation protocol while incorporating architectural modifications to replace the Linear Transformer component with xLSTM blocks. Multiple sample lengths are tested to assess model performance across different temporal contexts, using standard evaluation metrics including Balanced Accuracy, Cohen's Kappa, and Weighted F1 Score.

1.2.3 Statistical Analysis and Visualization

Beyond basic performance comparison, this thesis employs rigorous statistical analysis to ensure robust conclusions:

- **Statistical Significance Testing:** T-tests and Wilcoxon signed-rank tests are conducted to compare the best-performing model against other configurations, determining which models perform within statistically significant ranges of the optimal solution.
- **Performance Trend Analysis:** Plots illustrate how evaluation metrics change across different sample lengths. This visualization approach enables identification of optimal temporal contexts and understanding of model stability across different input durations.

- **Confusion Matrix Analysis:** The best-performing model configurations are evaluated again on the test set to generate detailed confusion matrices. This analysis reveals which classes benefit most from the architectural improvements and identifies specific classification challenges that different xLSTM variants address.

These analytical approaches provide both quantitative validation of performance improvements and qualitative insights into the behavioral differences between xLSTM variants and the baseline Linear Transformer architecture.

1.2.4 Dataset and Scope

This evaluation focuses on the Temple University EEG Event Corpus (TUEV) dataset, a subset of the Temple University Hospital EEG Corpus that provides a standardized benchmark for EEG event classification tasks [10]. The TUEV dataset was specifically designed to address the challenges of automated EEG event detection and classification in clinical settings.

Dataset Characteristics

The TUEV dataset contains EEG recordings with annotations for six distinct event types:

- **Spike and Slow Wave (SPSW):** Characteristic epileptiform discharges
- **Generalized Periodic Epileptiform Discharge (GPED):** Widespread periodic patterns
- **Periodic Lateralized Epileptiform Discharge (PLED):** Localized periodic patterns
- **Eye Movement (EYEM):** Ocular artifacts in EEG recordings
- **Artifact (ARTF):** Various technical and physiological artifacts
- **Background (BCKG):** Normal EEG background activity

The dataset employs a standard ACNS TCP montage with 16 channels, derived from bipolar combinations of EEG electrodes following the international 10-20 system. This montage configuration is widely used in clinical seizure analysis and provides spatial coverage across frontal, temporal, central, and occipital brain regions.

Data Organization and Scope

The TUEV corpus is divided into training and evaluation sets, with 359 training files and 159 evaluation files. The distribution of events varies across categories, with artifact and background events being most prevalent, while spike and slow wave events are less frequent. This class imbalance reflects the natural distribution of EEG events in clinical recordings, where pathological events are typically less frequent than normal background activity.

Each EEG sample is processed to extract event-centered windows, where the event of interest is positioned at the center of the temporal window. The original recordings are sampled at 250 Hz and can be resampled to other frequencies (such as 200 Hz) depending on experimental requirements. The temporal windows are extracted with configurable durations before and after the event, allowing for analysis of different temporal contexts.

The dataset’s characteristics align with typical clinical EEG recording scenarios, ensuring that findings are relevant to practical biosignal processing applications. The multi-class nature of the classification task, combined with the clinical relevance of the event types, makes TUEV an appropriate benchmark for evaluating the effectiveness of different sequence modeling architectures in biosignal analysis.

1.3 Outline

This thesis is structured as follows:

Chapter 2: Background and Related Work provides the theoretical foundation for this research. It covers the basics of biosignal processing, reviews traditional and modern deep learning approaches for EEG analysis, and introduces the BIOT architecture. The chapter also presents the xLSTM family of architectures, explaining their main innovations and advantages over traditional LSTMs.

Chapter 3: Methodology describes the experimental design and implementation. This chapter explains how xLSTM components are integrated into the BIOT framework, describes the modifications made to the original architecture, and outlines the experimental protocol. It also provides details about the TUEV dataset, preprocessing steps, and evaluation metrics used in this study.

Chapter 4: Experimental Setup presents the specific configuration parameters, hyperparameter settings, and implementation details. This chapter explains how the evaluation differs from the original BIOT protocol and justifies the experimental choices made to ensure fair comparison between architectures.

Chapter 5: Results and Analysis presents the experimental findings and analyzes the performance differences between xLSTM variants and the baseline Linear Transformer. The chapter includes statistical analysis, performance comparisons across different sample lengths, and detailed examination of model behavior.

Chapter 6: Discussion interprets the results in relation to existing literature, discusses what the findings mean for biosignal processing, and addresses the limitations of this study. This chapter also explores the practical relevance of the performance improvements and their potential use in clinical applications.

Chapter 7: Conclusion summarizes the key findings, highlights the contributions of this work to biosignal processing, and suggests directions for future research. The chapter also reflects on the broader implications of combining modern sequence modeling architectures with specialized biosignal processing frameworks.

Chapter 2

Background and Related Work

2.1 Biosignal Processing and BIOT Architecture

Biosignal processing is a central challenge in biomedical engineering. The main task is to analyze multi-channel time series data and extract clinically relevant information. Traditional approaches relied on handcrafted features and domain-specific preprocessing techniques. With the development of deep learning, more advanced and automated approaches have become possible.

2.1.1 Challenges in Biosignal Processing

Biosignals, such as EEG and Electrocardiogram (ECG), are multi-channel time series recorded at high sampling rates in various healthcare domains. These signals show several characteristics that make automated analysis challenging:

- **Format Heterogeneity:** Different datasets use different channel configurations, sampling rates, and recording durations [20]. For example, seizure detection datasets may use 16 channels at 256 Hz for 10-second recordings, while sleep analysis datasets often use different montages and sampling frequencies.
- **Missing Data:** Real-world recordings often have missing channels or corrupted segments [1]. This happens because of electrode displacement, amplifier saturation, or recording interruptions.
- **Variable Length:** Clinical recordings vary in duration depending on the application and clinical protocol [20]. This makes it difficult to use models that expect fixed input dimensions.
- **Cross-Dataset Generalization:** Models trained on one dataset often fail to generalize to data from different institutions, recording systems, or patient populations due to domain shift [20].

2.1.2 BIOT: Biosignal Transformer Architecture

The Biosignal Transformer (BIOT) architecture was developed to address these challenges [20]. It introduces a unified framework that can handle different biosignal formats. BIOT is inspired by the Vision Transformer (ViT) [5] and Audio Spectrogram Transformer (AST) [8], and adapts their tokenization concept to biosignal data.

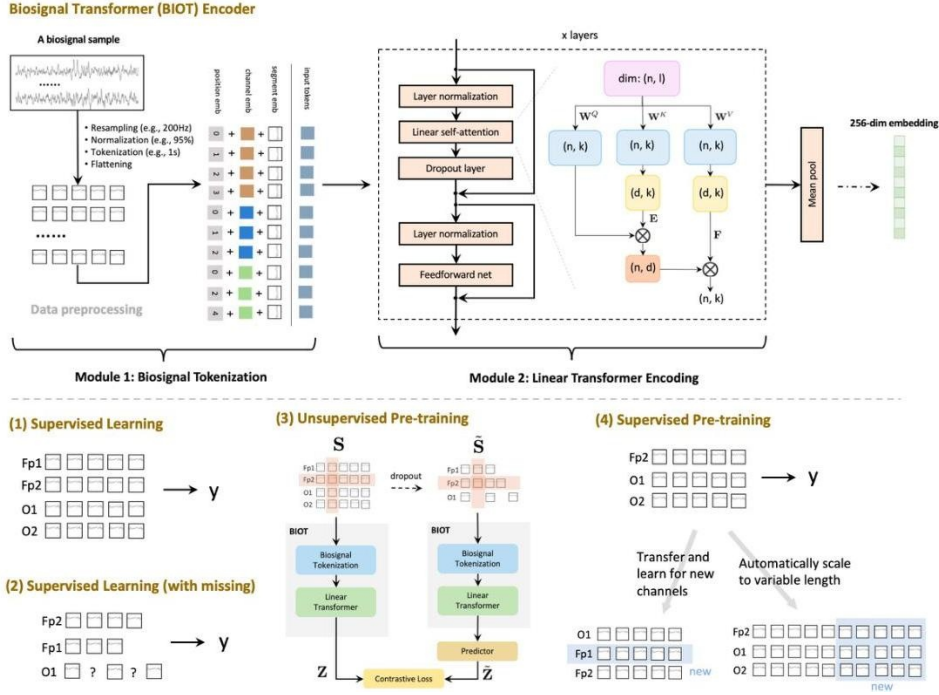


Figure 2.1: BIOT Architecture Overview. Upper: Biosignal tokenization module transforms heterogeneous biosignals into unified "sentences", followed by linear transformer processing. Lower: BIOT's versatility across different learning scenarios including supervised learning, missing data handling, and pre-training approaches. [20]

Biosignal Tokenization Module

The main innovation of BIOT is its biosignal tokenization approach. It transforms arbitrary biosignals into consistent "sentence" structures through several key steps:

Resampling and Normalization All biosignal data is first resampled to a common sampling rate (typically 200 Hz for EEG applications) using linear interpolation [20]. This rate selection follows the Nyquist-Shannon sampling theorem, ensuring adequate capture of frequencies up to 100 Hz, which covers the clinically relevant EEG frequency bands. The resampling step is critical for enabling cross-dataset learning, as different recording systems may use sampling rates ranging from 125 Hz to 512 Hz.

Following resampling, each channel is independently normalized using the 95th percentile of absolute amplitude values [20]:

$$S_{\text{norm}}[i] = \frac{S[i]}{\text{percentile}_{95}(|S[i]|)}$$

This percentile-based approach provides robustness to outliers and extreme voltage spikes (common artifacts in EEG recordings) while preserving the relative amplitude relationships within each channel. Unlike z-score normalization, it does not assume Gaussian distributions and maintains the temporal dynamics of the signal.

STFT-based Tokenization The core innovation of BIOT's tokenization is the transformation from time-domain samples to frequency-domain tokens. Unlike previous ap-

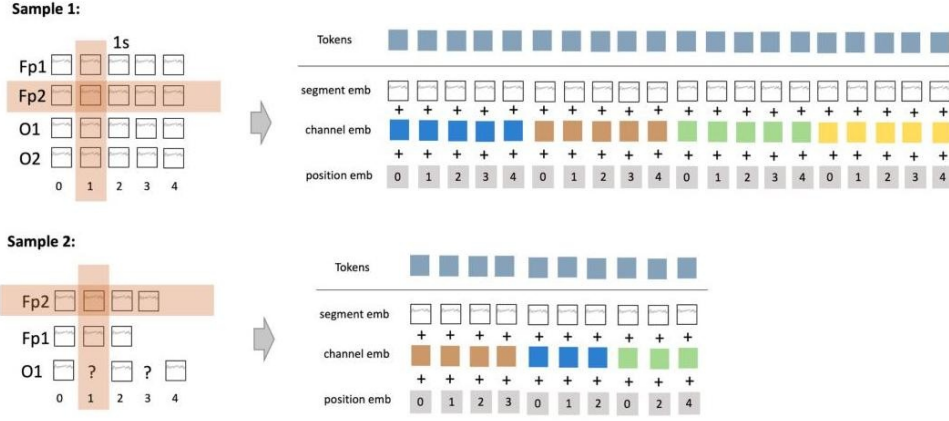


Figure 2.2: Biosignal Tokenization Process. Sample 1 demonstrates standard tokenization with four channels (Fp1, Fp2, O1, O2) over 5 seconds. Sample 2 illustrates BIOT’s flexibility with mismatched channels (missing O2), variable lengths, and missing values in O1. Different colors represent different channels, and the orange area shows spatial-temporal relationships between tokens. [20]

proaches that segment raw time-series data, BIOT applies the Short-Time Fourier Transform (STFT) to convert temporal segments into spectral representations [20].

For a biosignal sample $\mathbf{S} \in \mathbb{R}^{I \times J}$ where I denotes the number of channels and J represents the temporal length in samples, the tokenization proceeds as follows:

Step 1 - Channel-wise Processing: Each channel is processed independently. For channel i , the STFT is applied with parameters:

- **FFT Size (N):** Typically 200 samples (1 second at 200 Hz)
- **Hop Length (H):** Typically 100 samples (0.5 second stride)
- **Window Function:** Hann window for spectral smoothing
- **Centering:** Disabled (`center=False`) to maintain exact temporal alignment

Step 2 - Frequency Spectrum Extraction: The STFT transforms each temporal window into a complex-valued frequency spectrum. Taking only the magnitude (absolute value) yields a real-valued representation:

$$\text{STFT}(\mathbf{S}[i]) \in \mathbb{R}^{F \times T}$$

where $F = N/2 + 1$ is the number of frequency bins (101 bins for $N = 200$, spanning 0-100 Hz) and T is the number of temporal frames.

Step 3 - Token Count Calculation: For a signal of length J samples, the number of tokens generated per channel is:

$$T = \left\lfloor \frac{J - N}{H} \right\rfloor + 1$$

For example, a 5-second EEG window (1000 samples at 200 Hz) produces:

$$T = \left\lfloor \frac{1000 - 200}{100} \right\rfloor + 1 = 9 \text{ tokens per channel}$$

With 16 channels, this yields $16 \times 9 = 144$ total tokens. This dramatic reduction from 1000 time-domain samples to 144 frequency-domain tokens makes the representation tractable for sequence models while preserving essential spectral information.

Advantages of Channel-wise Processing: Unlike previous approaches that segment all channels simultaneously, BIOT’s independent channel processing provides several benefits [20]:

- **Missing Channel Robustness:** Absent channels are simply omitted from the sequence without requiring imputation or padding
- **Variable Length Handling:** Different channels can have different temporal extents (e.g., due to recording interruptions) without structural conflicts
- **Corrupted Segment Handling:** Individual tokens containing artifacts can be dropped without affecting neighboring channels

Token Embedding Each frequency-domain token must be transformed into a fixed-dimensional embedding suitable for sequence model processing. BIOT achieves this through three complementary embedding components that capture spectral content, spatial location, and temporal order (see Figure 2.2).

1. Segment Embedding - Frequency Content Representation

The STFT output for each token is a frequency spectrum vector $\mathbf{f} \in \mathbb{R}^F$ where $F = N/2 + 1$ (e.g., 101 frequency bins for $N = 200$). This vector represents the magnitude of each frequency component from 0 Hz to the Nyquist frequency (100 Hz at 200 Hz sampling rate).

The PatchFrequencyEmbedding module projects this frequency vector into a higher-dimensional embedding space through a learned linear transformation:

$$\mathbf{e}_{\text{segment}} = \mathbf{W}_{\text{freq}} \mathbf{f} + \mathbf{b}_{\text{freq}}$$

where $\mathbf{W}_{\text{freq}} \in \mathbb{R}^{d \times F}$ and $\mathbf{b}_{\text{freq}} \in \mathbb{R}^d$ are learnable parameters, and d is the embedding dimension (typically 256).

This projection serves multiple purposes:

- **Dimensionality Expansion:** Maps compact frequency representations (101-D) to richer embedding space (256-D)
- **Feature Learning:** Learns task-relevant frequency patterns (e.g., alpha rhythm for sleep stages, spike patterns for seizures)
- **Abstraction:** Creates representations that combine multiple frequency bands into semantically meaningful features

2. Channel Embedding - Spatial Information Encoding

Spatial information about electrode locations is crucial for EEG interpretation. Different channels capture activity from distinct brain regions, and their anatomical relationships inform clinical analysis. BIOT maintains a learnable embedding table:

$$\mathbf{E}_{\text{channel}} \in \mathbb{R}^{C \times d}$$

where C is the maximum number of channels supported (e.g., 16 for standard bipolar montage, but can be larger to accommodate diverse datasets).

For a token from channel i , the channel embedding $\mathbf{e}_{\text{channel}}^{(i)} = \mathbf{E}_{\text{channel}}[i]$ is retrieved and added to the segment embedding. This additive approach allows the model to learn channel-specific patterns while maintaining the spectral information from the segment embedding.

The learnable nature of channel embeddings enables the model to:

- Discover functional relationships between channels (e.g., frontal-parietal connectivity)
- Weight channels by their clinical relevance for specific tasks
- Generalize across datasets with partially overlapping channel configurations

3. Positional Embedding - Temporal Order Preservation

Within each channel, the temporal order of tokens is critical. A spike occurring before a slow wave has different clinical significance than the reverse order. BIOT employs sinusoidal positional encodings [19] that do not require learned parameters:

$$\text{PE}(\text{pos}, 2j) = \sin\left(\frac{\text{pos}}{10000^{2j/d}}\right) \quad (2.1)$$

$$\text{PE}(\text{pos}, 2j + 1) = \cos\left(\frac{\text{pos}}{10000^{2j/d}}\right) \quad (2.2)$$

where pos is the token position within its channel, j indexes the embedding dimensions, and d is the embedding size. These smooth, periodic functions enable the model to:

- Interpolate to sequence lengths not seen during training
- Learn relative temporal relationships through attention mechanisms
- Maintain consistent temporal encoding across different channels

Unified Token Representation

The final token embedding combines all three components through element-wise addition:

$$\mathbf{e}_{\text{token}}^{(i,t)} = \mathbf{e}_{\text{segment}}^{(i,t)} + \mathbf{e}_{\text{channel}}^{(i)} + \mathbf{e}_{\text{pos}}^{(t)}$$

where i indexes the channel and t indexes the temporal position.

After embedding, dropout is applied for regularization, yielding the final token representation ready for sequence model processing [20].

Token Sequence Formation

After embedding, tokens from all channels are concatenated (flattened) to form a single unified sequence. For a sample with I channels and T tokens per channel, the resulting sequence has length $N = I \times T$. For example, a 16-channel EEG segment producing 9 tokens per channel yields a 144-token sequence.

This flattening strategy treats the biosignal as a "sentence" where each token is a "word" [20].

Sequence Aggregation

After processing the token sequence through encoder blocks (Linear Transformer or xLSTM), the model produces contextualized representations $\mathbf{H} \in \mathbb{R}^{N \times d}$ where each token's representation has been enriched through interactions with other tokens.

To obtain a fixed-dimensional representation for the entire sample, BIOT employs mean pooling:

$$\mathbf{z} = \frac{1}{N} \sum_{n=1}^N \mathbf{H}[n]$$

The [CLS] token approach (prepending a special classification token, as used in BERT) was evaluated but found less effective for biosignals [20]. This may be because biosignal patterns are distributed across the entire sequence rather than concentrated at specific positions, making global averaging more appropriate than single-token representations.

The aggregated representation $\mathbf{z} \in \mathbb{R}^d$ is then passed to a classification head consisting of an Exponential Linear Unit (ELU) activation and linear layer for final predictions [20].

Applications and Capabilities

BIOT supports multiple learning paradigms:

- **Supervised Learning:** Standard classification tasks on complete datasets
- **Missing Data Handling:** Robust performance with incomplete channel configurations
- **Unsupervised Pre-training:** Large-scale pre-training on unlabeled biosignal data using a self-supervised approach where channels and tokens are randomly dropped. The perturbed signal is encoded by the same BIOT network, and a predictor head is applied to the perturbed branch. A contrastive loss is computed between the normalized embeddings of the original and perturbed signals.
- **Cross-dataset Transfer:** Knowledge transfer between datasets with different formats

2.2 Transformer Architecture and Linear Attention

The Transformer architecture revolutionized sequence modeling by introducing the attention mechanism as the main computational building block [19]. To understand why alternative architectures like xLSTM were developed, it is important to understand both the strengths and limitations of Transformers.

2.2.1 Self-Attention Mechanism

The core of the Transformer architecture is the scaled dot-product attention mechanism. For an input sequence $\mathbf{X} \in \mathbb{R}^{N \times d}$, it computes three matrices:

$$\mathbf{Q} = \mathbf{XW}^Q \tag{2.3}$$

$$\mathbf{K} = \mathbf{XW}^K \tag{2.4}$$

$$\mathbf{V} = \mathbf{XW}^V \tag{2.5}$$

where $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d_k}$ are learned parameter matrices for queries, keys, and values respectively.

The attention output is computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V} \quad (2.6)$$

This mechanism allows each position to attend to all positions in the sequence, enabling the model to capture long-range dependencies efficiently.

2.2.2 Computational Complexity Challenges

The standard attention mechanism has quadratic computational complexity $O(N^2)$ with respect to sequence length N . For biosignals, this becomes a problem. Multiple channels and high temporal resolution create long token sequences. For example, a 64-channel EEG recording of 20 seconds can generate over 1,280 tokens. This results in attention matrices with over 1.6 million elements, which is computationally very expensive.

2.2.3 Linear Attention in BIOT

To address scalability concerns, BIOT employs linear attention mechanisms that reduce computational complexity from $O(N^2)$ to $O(Nd)$ [20]. Following the family of linear attention methods [15], BIOT uses a rank- d approximation for the softmax attention by projecting keys and values with reduced-rank parameter matrices $\mathbf{E}^\top \in \mathbb{R}^{N \times d}$ and $\mathbf{F} \in \mathbb{R}^{d \times N}$ (where $d \ll N$). The formulation is:

$$\mathbf{H} = \text{softmax} \left(\frac{(\mathbf{X}\mathbf{W}^Q)(\mathbf{E}\mathbf{X}\mathbf{W}^K)^\top}{\sqrt{l_2}} \right) (\mathbf{F}\mathbf{X}\mathbf{W}^V) \quad (2.7)$$

where $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{l_1 \times l_2}$ are the query, key, and value projection matrices.

This approximation retains the main properties of attention while significantly reducing computational requirements, making it practical for long biosignal sequences with multiple channels.

2.2.4 Transformer in Biosignal Context

The application of Transformers to biosignals leverages several key advantages [19, 20]:

- **Flexible Sequence Handling:** Attention mechanisms can handle variable-length sequences and missing tokens naturally [20]. While Transformers are not truly permutation-invariant (positional encodings preserve temporal order), they provide flexibility in processing sequences of different lengths.
- **Long-range Dependencies:** Unlike RNNs, Transformers can capture dependencies across arbitrary distances without gradient degradation [19].
- **Parallelization:** The attention mechanism allows for efficient parallel computation during training [19].
- **Interpretability:** Attention weights provide insights into which temporal and spatial regions contribute to classification decisions.

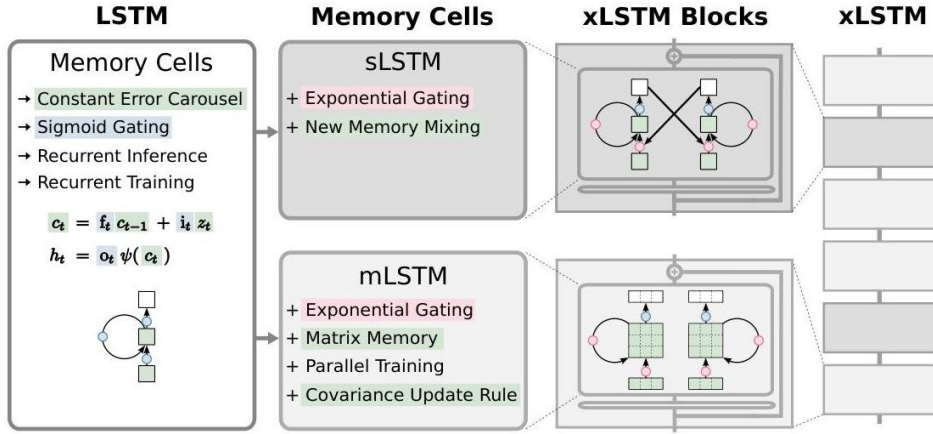


Figure 2.3: The Extended LSTM (xLSTM) Family. From left to right: (1) Original LSTM with constant error carousel and gating, (2) New sLSTM and mLSTM cells with exponential gating, (3) xLSTM blocks incorporating residual connections, (4) Complete xLSTM architecture with stacked blocks. [2]

2.3 Extended Long Short-Term Memory (xLSTM)

While Transformers have dominated sequence modeling in recent years, recent work has revisited LSTM architectures [13]. The Extended Long Short-Term Memory (xLSTM) family represents a significant advancement in recurrent neural network design [2]. It specifically addresses three main limitations of traditional LSTMs.

2.3.1 Limitations of Traditional LSTMs

Traditional LSTMs [13] face several constraints that limit their performance in modern applications [2]:

1. **Inability to Revise Storage Decisions:** Once information is stored in the cell state, LSTMs struggle to replace it with more relevant information encountered later in the sequence.
2. **Limited Storage Capacity:** Information must be compressed into scalar cell states, limiting the model's ability to maintain multiple pieces of information simultaneously.
3. **Sequential Processing:** The recurrent nature of LSTMs prevents parallelization during training, making them computationally inefficient compared to Transformers.

2.3.2 xLSTM Innovations

xLSTM addresses these limitations through two key innovations: exponential gating and novel memory structures.

Exponential Gating

Traditional LSTM gates use sigmoid activation functions, which can lead to vanishing gradients and limited expressiveness. xLSTM introduces exponential gating with suitable normalization:

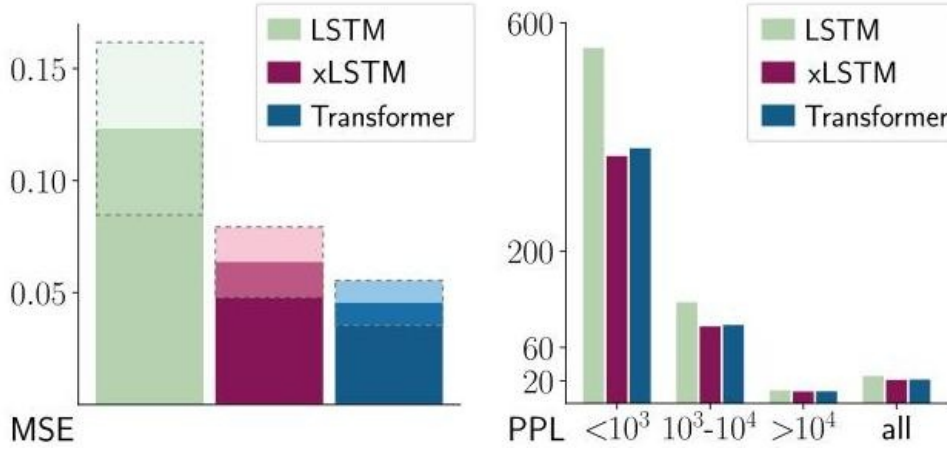


Figure 2.4: LSTM Limitations Illustrated. Left: Nearest Neighbor Search problem showing LSTM’s inability to revise storage decisions (higher MSE indicates worse performance). Right: Rare Token Prediction demonstrating limited storage capacity (higher perplexity indicates worse performance on rare tokens). xLSTM addresses both limitations through exponential gating and matrix memory. [2]

$$\tilde{f}_t = \exp(f_t), \quad \tilde{i}_t = \exp(i_t)$$

where f_t and i_t are the forget and input gate pre-activations. This exponential formulation allows more dynamic gating behavior and better gradient flow.

Stabilization: Exponential activation functions can lead to numerical overflow. To address this, xLSTM employs a log-domain stabilization technique using an additional state m_t that tracks the running maximum:

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)) \quad (2.8)$$

$$i'_t = \exp(\log(i_t) - m_t) = \exp(\tilde{i}_t - m_t) \quad (2.9)$$

$$f'_t = \exp(\log(f_t) + m_{t-1} - m_t) \quad (2.10)$$

The stabilized gates f'_t and i'_t are then used in the forward pass, ensuring numerical stability without changing the model’s outputs or gradients.

Memory Structure Innovations

xLSTM introduces two distinct memory architectures:

Scalar LSTM (sLSTM) sLSTM retains the scalar memory structure of traditional LSTMs but introduces exponential gating to enable dynamic storage revision. The key innovation is exponential activation functions for gates with appropriate normalization. The sLSTM update equations are:

$$\mathbf{c}_t = \tilde{\mathbf{f}}_t \odot \mathbf{c}_{t-1} + \tilde{\mathbf{i}}_t \odot \mathbf{z}_t \quad (2.11)$$

$$\mathbf{n}_t = \tilde{\mathbf{f}}_t \odot \mathbf{n}_{t-1} + \tilde{\mathbf{i}}_t \quad (2.12)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \frac{\mathbf{c}_t}{\mathbf{n}_t} \quad (2.13)$$

where $\tilde{\mathbf{f}}_t = \exp(\mathbf{f}_t)$ and $\tilde{\mathbf{i}}_t = \exp(\mathbf{i}_t)$ are the exponential forget and input gates, and $\mathbf{n}_t$ is a normalizer state that tracks the cumulative effect of gates. The exponential gating allows more effective memory management and the ability to revise storage decisions dynamically, addressing a key limitation of traditional LSTMs.

Matrix LSTM (mLSTM) mLSTM represents a major departure from scalar memory, using matrix-valued cell states $\mathbf{C}_t \in \mathbb{R}^{d \times d}$ to enhance storage capacity. This architecture enables:

- **Enhanced Storage Capacity:** Matrix memory can maintain multiple independent pieces of information simultaneously through key-value associations.
- **Parallelizable Computation:** Unlike sLSTM, the matrix formulation is fully parallelizable as it abandons memory mixing (no hidden-hidden recurrent connections).
- **Covariance Updates:** The outer product update rule based on key-value pairs provides richer representational capabilities.

The mLSTM update equations with normalizer state are:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \mathbf{v}_t \mathbf{k}_t^\top \quad (2.14)$$

$$\mathbf{n}_t = \mathbf{f}_t \odot \mathbf{n}_{t-1} + \mathbf{i}_t \odot \mathbf{k}_t \quad (2.15)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \frac{\mathbf{C}_t \mathbf{q}_t}{\max\{\mathbf{n}_t^\top \mathbf{q}_t, 1\}} \quad (2.16)$$

where $\mathbf{f}_t$, $\mathbf{i}_t$, and $\mathbf{o}_t$ are exponential gates, $\mathbf{v}_t$, $\mathbf{k}_t$, and $\mathbf{q}_t$ are value, key, and query vectors, and $\mathbf{n}_t$ is the normalizer state. The normalizer state tracks the weighted sum of key vectors and ensures stable retrieval by preventing division by near-zero values.

2.3.3 xLSTM Block Architecture

xLSTM components are integrated into residual block architectures, similar to Transformer blocks. Each xLSTM block contains:

- Layer normalization for stable training
- Residual connections for gradient flow
- Feed-forward networks for non-linear transformations
- Dropout for regularization

Multiple xLSTM blocks can be stacked to create deep architectures that combine the benefits of both sLSTM and mLSTM components.

2.3.4 Theoretical Advantages for Biosignals

The xLSTM architecture offers several theoretical advantages for biosignal processing:

- **Enhanced Temporal Modeling:** Exponential gating allows more nuanced temporal dependency capture, crucial for biosignals where precise timing of events is critical for clinical interpretation.
- **Improved Memory Management:** The ability to revise storage decisions is particularly valuable for long biosignal sequences where later information may override earlier observations, addressing a key limitation that traditional LSTMs face.
- **Scalable Architecture:** mLSTM's fully parallelizable nature (due to the absence of memory mixing) addresses computational efficiency concerns while maintaining recurrent modeling benefits. In contrast, sLSTM remains inherently sequential due to memory mixing, though efficient CUDA implementations mitigate this limitation.
- **Rich Representations:** Matrix memory structures can potentially capture complex inter-channel relationships in multi-channel biosignals through key-value associations, providing enhanced storage capacity compared to scalar memories.
- **State Tracking Capability:** Unlike Transformers and State Space Models, xLSTM (particularly sLSTM with memory mixing) can solve state tracking problems, which may be important for tracking evolving patterns in biosignal sequences.

2.4 Integration of xLSTM with BIOT

Integrating xLSTM architectures into the BIOT framework is a natural next step that combines the strengths of both approaches. This section describes the theoretical foundation for this integration and its potential benefits.

2.4.1 Architectural Compatibility

BIOT's modular design enables straightforward integration of xLSTM components as alternatives to the Linear Transformer encoder. The integration maintains the complete biosignal tokenization pipeline:

1. **Tokenization Module:** The resampling, normalization, STFT-based segmentation, and embedding components remain unchanged, preserving BIOT's ability to handle heterogeneous biosignal formats.
2. **Sequence Processing:** The Linear Transformer blocks are replaced with xLSTM block stacks (sLSTM, mLSTM, or hybrid combinations), enabling evaluation of different memory mechanisms.
3. **Aggregation:** Sequence representations are aggregated via mean pooling across tokens before classification, consistent with the original BIOT approach.

This modular replacement allows systematic comparison of sequence modeling approaches while preserving the key advantages of BIOT's flexible tokenization strategy.

2.4.2 Theoretical Synergies

Combining BIOT tokenization with xLSTM processing offers several theoretical advantages:

- **Preserved Flexibility:** BIOT's tokenization approach keeps compatibility with different biosignal formats while xLSTM provides enhanced sequence processing.
- **Enhanced Temporal Modeling:** xLSTM's superior memory mechanisms could better capture the complex temporal patterns in biosignals.
- **Scalable Performance:** The combination maintains computational efficiency while potentially improving classification accuracy.
- **Cross-dataset Applicability:** The integration preserves BIOT's ability to work across different datasets while potentially improving performance through better sequence modeling.

2.4.3 Expected Benefits

From a theoretical perspective, integrating xLSTM with BIOT could address several limitations:

- **Improved Long-range Dependencies:** xLSTM's enhanced memory capabilities could better capture dependencies across long biosignal sequences.
- **Better Pattern Recognition:** The matrix memory structures in mLSTM could enable more sophisticated pattern recognition in multi-channel biosignals.
- **Enhanced Robustness:** xLSTM's ability to revise storage decisions could improve robustness to noise and artifacts that are common in biosignals.

This theoretical foundation is why this thesis evaluates xLSTM variants within the BIOT framework, which is the focus of the following experimental chapters.

Chapter 3

Methodology

This chapter presents the experimental methodology used to evaluate xLSTM architectures within the BIOT framework for EEG event classification. The focus is on the conceptual framework, scientific rationale, and research design that guides the experimental evaluation.

3.1 Experimental Framework

The experimental framework builds upon the existing BIOT architecture by integrating xLSTM components as alternatives to the traditional Transformer encoder layers. This integration aims to evaluate whether the enhanced memory mechanisms of xLSTM can improve the model's ability to capture long-range dependencies in EEG sequences while maintaining computational efficiency.

3.1.1 Research Questions and Objectives

The primary research questions guiding this experimental evaluation are:

1. **Performance Comparison:** How do xLSTM variants (sLSTM, mLSTM, and their combination) compare to the original BIOT architecture for EEG event classification?
2. **Temporal Context:** What is the optimal temporal window size for capturing relevant context around clinical EEG events?
3. **Architectural Components:** What are the individual contributions of sLSTM and mLSTM components to overall classification performance?
4. **Robustness:** How consistent is model performance across different random initializations and class distributions?

These questions are addressed through systematic experimentation with controlled parameter variations, enabling clear attribution of performance differences to specific architectural choices.

3.1.2 Integration Strategy

The integration of xLSTM into the BIOT framework preserves the core tokenization and preprocessing mechanisms while replacing the sequence modeling components. This modular approach enables fair comparison by isolating the impact of different sequence modeling architectures.

Key integration principles include:

- **Tokenization Preservation:** The biosignal tokenization module remains unchanged, ensuring all models process identical input representations
- **Dimensional Consistency:** All encoder variants maintain the same embedding dimensions and token lengths
- **Architectural Modularity:** The encoder component can be replaced without modifying preprocessing or classification layers
- **Compatibility:** Both sLSTM and mLSTM variants operate on the same tokenized input format

This integration strategy maintains the advantages of BIOT’s flexible tokenization while enabling systematic evaluation of different temporal sequence modeling approaches.

3.1.3 Model Variants

Four distinct model variants are evaluated to systematically assess the impact of different xLSTM components:

BIOT Baseline: The original BIOT architecture with Linear Attention Transformer serves as the baseline for comparison. This configuration provides the reference performance against which xLSTM enhancements are measured.

sLSTM: This configuration replaces the Transformer encoder with sLSTM blocks featuring scalar memory mechanisms and exponential gating. The sLSTM variant emphasizes sequential processing with memory mixing, potentially improving the model’s ability to track evolving patterns in EEG sequences.

mLSTM: This configuration employs mLSTM blocks with matrix memory structures, enabling enhanced storage capacity through key-value associations. The mLSTM variant is fully parallelizable, offering computational efficiency while maintaining rich representational capabilities.

sLSTM + mLSTM: The combined configuration integrates both memory mechanisms in a unified architecture, leveraging the complementary strengths of scalar and matrix memory. This variant represents the most complex model with the highest representational capacity.

Each variant is evaluated under identical training conditions and data processing pipelines, enabling direct performance attribution to the architectural differences in sequence modeling.

3.2 Dataset: Temple University Hospital EEG Event Corpus

3.2.1 Background and Motivation

Clinical EEG data represents an invaluable but underutilized resource for machine learning research. While hundreds of thousands of EEG recordings are collected annually in hospitals worldwide, relatively few are available to the research community in formats suitable for algorithm development [17]. This scarcity is particularly problematic given that modern deep learning approaches require massive amounts of diverse training data to achieve robust performance.

The Temple University Hospital (TUH) EEG Corpus was developed to address this gap, providing the research community with access to large-scale clinical EEG data. The complete TUH corpus comprises 16,986 sessions from 10,874 unique subjects collected between 2002 and the present, representing approximately 29.1 years of cumulative EEG data [17]. Importantly, this is clinical-grade data collected in real-world hospital settings including the Intensive Care Unit (ICU), epilepsy monitoring unit, emergency room, and outpatient services, rather than tightly controlled laboratory conditions.

The TUEV subset was created specifically for event classification tasks and contains sessions known to include clinically significant events such as epileptiform discharges, artifacts, and eye movements [10]. This focused subset enables targeted development and evaluation of algorithms for automatic event detection and classification, a critical capability for assisting neurologists in analyzing long-term EEG recordings.

3.2.2 Dataset Structure and Organization

The TUEV dataset is organized into two primary partitions: a training set for model development and an evaluation set for independent testing. The training set contains 359 EEG files, while the evaluation set comprises 159 files. This separation ensures that models can be evaluated on data from sessions not seen during training.

Signal data is stored in the European Data Format (EDF), an open-source format that encodes multi-channel time-series data using 16-bit integers. Each EDF file contains both the raw EEG signals and associated metadata in an ASCII header describing channel labels, sampling rates, and recording parameters. The signal data represents pruned recordings where sections marked as uninformative by attending technicians were removed, resulting in focused segments containing relevant neural activity.

The file organization follows a hierarchical structure that preserves relationships between recordings. For the evaluation set, files are organized by session index (e.g., `eval/032/bckg_032_a_.edf`), where multiple files may represent segments from the same recording session. For the training set, files are indexed by patient identifier (e.g., `train/00002275/00002275_00000001.edf`), allowing sessions from the same patient to be identified and grouped appropriately during data partitioning.

3.2.3 Event Classes and Clinical Significance

The TUEV dataset annotates six distinct classes of events, three representing clinically significant neural patterns and three representing various forms of background activity or noise:

Clinical Event Classes:

- **Spike and Sharp Wave (SPSW):** Epileptiform transients characterized by sharp waveforms typically observed in patients with epilepsy. These events represent abnormal synchronized neural activity and are critical markers for seizure disorders.
- **Generalized Periodic Epileptiform Discharge (GPED):** Periodic discharges that manifest as repetitive patterns distributed across multiple brain regions. These can appear as periodic short-interval diffuse discharges, periodic long-interval diffuse discharges, or suppression-burst patterns, and may indicate severe brain dysfunction.
- **Periodic Lateralized Epileptiform Discharge (PLED):** Lateralized periodic discharges that appear repetitively over one hemisphere. These quasi-periodic spike

or sharp wave patterns are indicative of focal brain abnormalities and often manifest in patients with acute neurological conditions.

Background and Noise Classes:

- **Eye Movement (EYEM):** Ocular artifacts generated by eye movements. These events can mimic spike patterns in frontal electrodes and must be distinguished from true epileptiform activity during clinical interpretation.
- **Artifact (ARTF):** Non-cerebral electrical activity originating from equipment, patient movement, muscle activity, or environmental interference. Artifact detection is essential for maintaining data quality and preventing false positives in clinical analysis.
- **Background (BCKG):** Background EEG activity that does not belong to any of the other five classes. This catch-all category represents normal or non-specific brain activity.

Event annotations are provided in two complementary formats. The `.rec` files contain comma-separated records specifying: channel number (0-21), start time in seconds, end time in seconds, and event class (1-6). The `.lab` files provide the same information with timestamps in units of 10 microseconds and four-letter class codes (spsw, gped, pled, eyem, artf, bckg). This dual format supports both machine parsing and human interpretation.

3.2.4 Class Distribution and Imbalance

The distribution of events varies significantly across the training and evaluation sets, reflecting the natural distribution of events in clinical practice. In the training set, background events are most prevalent (211 files), followed by artifacts (164 files), while clinical events like SPSW are less common (27 files). The evaluation set shows similar patterns with artifacts in 46 files and SPSW in only 9 files.

This class imbalance presents a significant challenge for machine learning algorithms, which tend to bias predictions toward majority classes. The imbalance necessitates:

- Evaluation metrics that account for class imbalance (balanced accuracy, Cohen's Kappa)
- Careful interpretation of overall accuracy metrics
- Potential class-specific performance analysis
- Consideration of clinical cost asymmetries (false negatives vs. false positives)

The imbalanced distribution reflects real-world clinical scenarios where pathological events are less frequent than normal background activity, making this dataset particularly valuable for developing clinically applicable algorithms.

3.2.5 Electrode Configuration and Channel System

Understanding the electrode configuration and channel labeling system is crucial for correctly interpreting EEG data from the TUEV dataset. The recordings follow standardized electrode placement protocols that enable consistent spatial localization of neural activity.

International 10-20 Electrode System: The TUEV recordings use the International 10-20 system, the most widely adopted standard for EEG electrode placement (see Figure 3.1). This system positions 21 electrodes on the scalp with spacing determined by 10% or 20% of the total distance from nasion (front of skull) to inion (back of skull) [6]. Electrode names follow a systematic convention where letters indicate brain region (Fp=prefrontal, F=frontal, C=central, T=temporal, P=parietal, O=occipital, A=auricular) and numbers indicate laterality (odd=left hemisphere, even=right hemisphere, Z=midline).

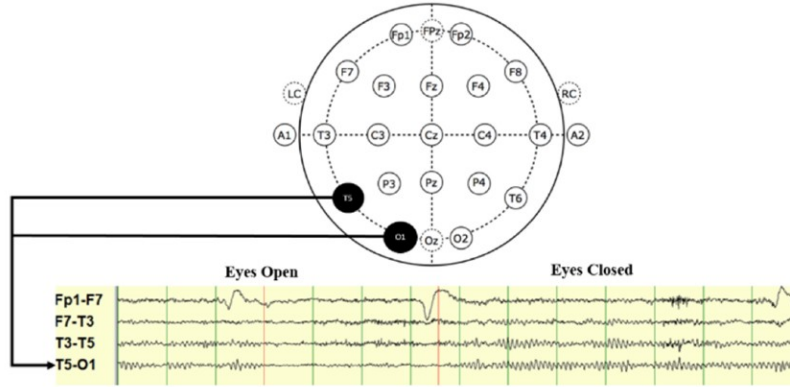


Figure 3.1: International 10-20 electrode system showing the standardized placement of electrodes on the scalp. The system uses anatomical landmarks (nasion, inion, and preauricular points) to ensure consistent electrode positioning across subjects.

Unipolar Recording Montages: Raw EEG signals are recorded as differential voltages between individual electrodes and a reference point, a process known as unipolar recording. The TUEV dataset contains recordings using two primary unipolar montages: Average Reference (AR) montage, which uses the average of multiple electrodes as the reference, and Linked Ears (LE) montage, which links the left and right ear electrodes (A1 and A2) to provide a stable reference point [6].

Channel Labels and Variability: Each channel in an EDF file is identified by a label such as “EEG FP1-REF” or “EEG C3-REF”, where the electrode name is followed by the reference type [6]. An important characteristic of clinical data is that channels do not appear in a guaranteed order within files. Different recording sessions may store the same electrode in different positions, requiring software to access channels by label rather than assuming fixed positions. The complete TUH corpus contains over 40 different channel configurations, reflecting the reality of data collected across different hospital units, equipment generations, and clinical protocols.

3.3 Data Preprocessing Pipeline

The preprocessing pipeline transforms raw EEG recordings into a format suitable for neural network training while preserving the temporal and spatial characteristics essential for event classification. This section describes the conceptual approach and rationale for each preprocessing stage.

3.3.1 Bipolar Montage Conversion

While the raw EEG data is recorded using unipolar montages with a common reference, neurologists typically analyze EEG signals using bipolar montages for clinical interpretation. A bipolar montage represents the voltage difference between pairs of adjacent electrodes, which provides several advantages:

- **Local Feature Enhancement:** Bipolar derivations accentuate local neural activity by measuring potential differences between nearby electrodes
- **Common-Mode Artifact Rejection:** Artifacts affecting large scalp regions simultaneously are reduced through differential measurement
- **Clinical Consistency:** Bipolar montages match the representation neurologists use for visual analysis, facilitating clinical validation
- **Spatial Localization:** Local voltage gradients provide better spatial resolution for identifying event origins

The preprocessing pipeline converts unipolar recordings to a standard Temporal Central Parasagittal (TCP) bipolar montage, also known as the double-banana montage due to its characteristic visualization pattern (see Figure 3.2). The TCP montage creates 22 channels by computing differences between adjacent electrodes along electrode chains.

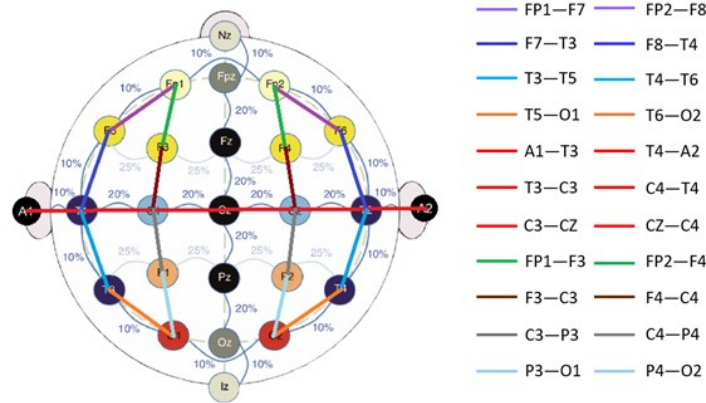


Figure 3.2: Temporal Central Parasagittal (TCP) montage showing the electrode chains used for bipolar derivations. The longitudinal chains traverse from front to back through temporal regions, while parasagittal chains pass near the midline. This montage is commonly referred to as the “double-banana” configuration.

Longitudinal Chains traverse from front to back across the temporal regions:

- Left temporal: FP1-F7, F7-T3, T3-T5, T5-O1
- Right temporal: FP2-F8, F8-T4, T4-T6, T6-O2

Parasagittal Chains traverse from front to back near the midline:

- Left parasagittal: FP1-F3, F3-C3, C3-P3, P3-O1
- Right parasagittal: FP2-F4, F4-C4, C4-P4, P4-O2

Central Chain crosses laterally across the midline:

- Transverse: A1-T3, T3-C3, C3-CZ, CZ-C4, C4-T4, T4-A2

For this work, 16 of these 22 channels are utilized, omitting the transverse chain components that include auricular electrodes. This selection provides comprehensive spatial coverage while maintaining consistency across recordings.

3.3.2 Signal Resampling

The raw EEG signals are recorded at varying sampling rates across different recording sessions. Signal resampling standardizes the temporal resolution across all recordings, enabling consistent temporal analysis. The resampling from the original 250 Hz to 200 Hz serves multiple purposes:

- **Standardization:** Ensures all signals have identical temporal resolution
- **Nyquist Criterion:** Maintains sufficient sampling for frequency content up to 100 Hz, covering the clinical frequency range of interest (0.5-50 Hz)
- **Computational Efficiency:** Reduces the number of time points while preserving clinically relevant information

The resampling operation employs anti-aliasing filters to prevent spectral distortion and artifacts, ensuring the integrity of frequency-domain features used in subsequent processing stages.

3.3.3 Signal Normalization

Each channel undergoes normalization by scaling to the 95th percentile of absolute signal amplitudes. This approach divides each channel's values by its 95th percentile, bringing all channels to a comparable amplitude range while maintaining robustness against extreme values.

The percentile-based normalization offers several advantages for clinical EEG data:

- **Robust to Outliers:** Large-amplitude artifacts or pathological events do not dominate the scaling factor, unlike mean-based or maximum-based approaches
- **Channel Consistency:** Different channels and recording sessions are normalized to comparable scales, facilitating cross-channel learning
- **Preserves Temporal Patterns:** The relative amplitude relationships within each channel remain intact, maintaining clinically relevant signal dynamics

This normalization strategy provides stable scaling across diverse recording conditions while preserving the amplitude patterns essential for distinguishing different event types.

3.3.4 Event Window Extraction

The event extraction process creates fixed-length windows centered around annotated events, enabling consistent input dimensions for neural network training while capturing relevant temporal context. This windowing approach addresses several key challenges:

Temporal Context Capture: Clinical interpretation of EEG events requires context around the annotated event. Activity preceding an event may represent buildup patterns, while post-event activity may indicate resolution or evolution. The configurable window approach enables systematic investigation of optimal context length.

Fixed-Size Input Requirement: Neural networks typically require fixed-size inputs for efficient batch processing. The windowing approach transforms variable-length annotated events into uniform representations suitable for modern deep learning frameworks.

Multi-channel Spatial Context: Events are extracted across all 16 bipolar channels, not just the annotated channel. This provides spatial context and enables the model to learn patterns that may involve multiple brain regions, improving classification robustness.

The window extraction strategy employs configurable parameters (`secondsBefore` and `secondsAfter`) that define the temporal extent around each event. This parameterized design enables systematic exploration of different temporal contexts through experimental variation, which is crucial for identifying the optimal window size for classification performance.

3.4 Experimental Design

3.4.1 Training Strategy

The training approach employs supervised learning with careful attention to data partitioning and validation strategy. The experimental design emphasizes reproducibility, robust evaluation, and systematic comparison across model variants.

Data Partitioning Philosophy: The dataset is partitioned into three distinct sets with clear purposes:

- **Training set:** Used for model parameter optimization through gradient descent
- **Validation set:** Used for hyperparameter tuning, model selection, and early stopping decisions
- **Test set:** Reserved for final unbiased performance assessment

Critically, the validation split is performed at the subject level rather than the sample level. This ensures that all samples from the same subject remain within the same partition, preventing data leakage that would artificially inflate performance estimates. Subject-level splitting provides more realistic assessment of model generalization to new patients, which is the ultimate goal in clinical applications.

Reproducibility Requirements: To enable reproducible research and facilitate comparison with future work:

- Fixed random seeds are applied consistently across all random number generators
- Deterministic algorithms are enabled where computational efficiency allows
- Data loading order is standardized through sorted file lists
- Subject-level splitting uses fixed random states for consistency

These measures ensure that reported results can be reliably reproduced and that performance differences reflect true architectural improvements rather than random initialization effects.

3.4.2 Evaluation Methodology

The evaluation methodology employs multiple metrics to provide comprehensive assessment of model performance across different aspects of the classification task. The metric selection addresses the specific challenges of clinical EEG event classification.

3.4.3 Evaluation Metrics

Primary Metric - Balanced Accuracy: Balanced accuracy serves as the primary evaluation metric due to the significant class imbalance in the TUEV dataset. Unlike standard accuracy, which can be misleadingly high when a model simply predicts the majority class, balanced accuracy computes the average recall across all classes:

$$\text{BACC} = \frac{1}{K} \sum_{i=1}^K \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i} \quad (3.1)$$

where K is the number of classes, TP_i denotes true positives for class i , and FN_i denotes false negatives. For the 6-class TUEV problem, each class contributes equally (16.67%) to the final score regardless of class frequency. This ensures that rare but clinically important events like SPSW receive equal weight to common events like artifacts.

Secondary Metrics: Additional metrics provide complementary perspectives on model performance:

Weighted F1-Score balances precision and recall while accounting for class frequency:

$$\text{F1}_{\text{weighted}} = \sum_{i=1}^K w_i \cdot \frac{2 \cdot \text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \quad (3.2)$$

where $w_i = \frac{n_i}{n_{\text{total}}}$ is the proportion of samples in class i . This metric is computed per-class then averaged with support-based weights.

Cohen's Kappa accounts for chance agreement, providing robust assessment beyond simple accuracy:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (3.3)$$

where p_o is observed agreement (accuracy) and p_e is expected agreement under independence:

$$p_e = \sum_{i=1}^K \frac{(\text{TP}_i + \text{FP}_i)(\text{TP}_i + \text{FN}_i)}{n_{\text{total}}^2} \quad (3.4)$$

AUC-PR Macro focuses on positive class performance through precision-recall curves, particularly important for rare clinical events:

$$\text{AUC-PR}_{\text{macro}} = \frac{1}{K} \sum_{i=1}^K \int_0^1 \text{Precision}_i(\text{Recall}_i) d\text{Recall}_i \quad (3.5)$$

One-vs-rest precision-recall curves are computed for each class and then macro-averaged.

Overall Accuracy provides intuitive performance understanding:

$$\text{Accuracy} = \frac{\sum_{i=1}^K \text{TP}_i}{n_{\text{total}}} \quad (3.6)$$

This standard classification metric enables direct comparison with published literature.

3.4.4 Validation Protocol

Training Monitoring: During training, the validation set is evaluated after every epoch to track model convergence and detect potential overfitting. The training data is split at the subject level, with 10% of training subjects allocated to the validation set. This subject-level splitting ensures that all samples from the same subject remain in the same partition, preventing data leakage and providing realistic evaluation of generalization performance.

Checkpointing Strategy: Two distinct checkpointing strategies are employed to capture different aspects of model performance. The first strategy monitors validation cross-entropy loss, saving the model whenever the loss decreases to a new minimum. This captures best probabilistic calibration. The second strategy tracks validation balanced accuracy, preserving the model whenever accuracy increases to a new maximum. This captures best classification performance. The dual approach enables analysis of the trade-off between loss minimization and classification accuracy.

Confusion Matrix Analysis: For each trained model, a confusion matrix is computed on the test set to provide detailed insight into classification patterns. The confusion matrix reveals which event classes are most frequently confused, enabling identification of systematic misclassification patterns. This is particularly valuable for understanding model behavior on clinically similar events (e.g., distinguishing epileptiform activity from artifacts).

Final Assessment: After training completion, both checkpoint variants (loss-based and accuracy-based) are evaluated on the held-out test set. All five metrics are computed to provide comprehensive performance characterization.

3.5 Statistical Analysis Framework

The multi-seed experimental design enables rigorous statistical analysis of performance differences between architectural configurations. This section describes the statistical framework used to interpret experimental results and assess the significance of performance differences.

3.5.1 Performance Aggregation

For each combination of model architecture and temporal window, performance metrics are aggregated across the five random seeds (10, 42, 123, 456, 999). The mean provides a central tendency estimate, while the standard deviation quantifies performance variability:

- **Mean Performance:** Indicates typical expected performance for a given configuration
- **Standard Deviation:** Quantifies sensitivity to random initialization and data shuffling

Configurations with low standard deviation demonstrate robust performance across initializations, while high variability suggests sensitivity to initialization or optimization challenges. This variability assessment is crucial for identifying architectures that perform reliably in deployment.

3.5.2 Statistical Significance Testing

To rigorously assess whether observed performance differences between architectural configurations are statistically meaningful, two complementary statistical tests are employed.

Paired t-Test: The paired t-test assesses whether the mean difference between two related samples differs significantly from zero. For comparing two model configurations across n random seeds, let X_i and Y_i denote the performance metric values for seeds $i = 1, \dots, n$. The test statistic is:

$$t = \frac{\bar{D}}{s_D / \sqrt{n}} \quad (3.7)$$

where $\bar{D} = \frac{1}{n} \sum_{i=1}^n (X_i - Y_i)$ is the mean difference and s_D is the standard deviation of differences. The paired design accounts for correlations across seeds due to shared data splits and training procedures. The t-test assumes that differences are approximately normally distributed, which is reasonable for aggregated metrics across multiple seeds.

Wilcoxon Signed-Rank Test: As a non-parametric alternative to the paired t-test, the Wilcoxon signed-rank test does not assume normality of the difference distribution. The test ranks the absolute differences $|X_i - Y_i|$ and assigns signs based on whether $X_i > Y_i$ or $X_i < Y_i$. The test statistic W is the sum of ranks for positive differences. This test is more robust to outliers and violations of normality assumptions, providing complementary evidence to the t-test. When both tests agree on significance, this strengthens confidence in the conclusion.

Significance Levels: Statistical significance is assessed at the conventional $\alpha = 0.05$ level. Results with $p < 0.05$ indicate strong evidence against the null hypothesis of no difference, while $p < 0.01$ indicates very strong evidence. Both tests are applied to each performance metric, enabling cross-validation of statistical conclusions and robust assessment of architectural differences.

3.5.3 Comparative Analysis

The experimental design enables several types of comparative analysis:

Architecture Comparison: Holding temporal window constant, performance differences across the four architectural variants reveal the impact of xLSTM components. This isolates the effect of different sequence modeling approaches.

Temporal Window Comparison: Holding architecture constant, performance trends across window sizes identify optimal temporal context requirements for event classification.

Component Attribution: Comparing sLSTM-only, mLSTM-only, and combined configurations reveals individual component contributions and potential synergistic effects.

The comprehensive experimental design provides the data necessary to answer all research questions posed at the beginning of this chapter, enabling clear conclusions about the utility of xLSTM integration for clinical EEG event classification.

Chapter 4

Experimental Setup

This chapter provides comprehensive technical specifications and implementation details for the experimental evaluation. While Chapter 3 described the conceptual framework and scientific rationale, this chapter focuses on the specific configurations, parameter values, and computational infrastructure used to execute the experiments.

4.1 Implementation Environment

4.1.1 Software Stack

The experimental implementation leverages modern deep learning frameworks and scientific computing libraries:

Core Deep Learning Framework:

- **PyTorch 2.x**: Primary deep learning framework providing automatic differentiation and GPU acceleration
- **PyTorch Lightning 2.x**: High-level framework for training organization, distributed computing, and logging infrastructure
- **CUDA 11.8+**: NVIDIA CUDA toolkit for GPU computation

xLSTM Integration:

- **xlstm package**: Official implementation providing optimized sLSTM and mLSTM components with CUDA backend support (<https://github.com/NX-AI/xlstm>)

Signal Processing and Data Handling:

- **NumPy 1.24+**: Numerical computing and array operations
- **SciPy 1.10+**: Signal processing for resampling and filtering
- **MNE-Python 1.3+**: EEG data loading and EDF format handling
- **PyHealth**: Medical data analysis utilities and evaluation metrics

Experiment Tracking and Visualization:

- **Weights & Biases (wandb)**: Comprehensive experiment tracking, metric logging, and model artifact management
- **Matplotlib/Seaborn**: Visualization for results analysis

4.1.2 Hardware Configuration

The experimental evaluation is conducted on a single-GPU workstation:

GPU Specifications:

- **Model:** NVIDIA GeForce RTX 3090
- **Memory:** 24GB GDDR6X
- **CUDA Cores:** 10,496
- **Tensor Cores:** 328 (3rd generation)
- **Memory Bandwidth:** 936 GB/s

The 24GB memory capacity enables processing of long EEG sequences and supports the memory-intensive operations required by matrix LSTM implementations. The substantial memory bandwidth facilitates efficient gradient computation during backpropagation through long sequences.

Training Strategy: Single-GPU training without distributed computing. While PyTorch Lightning supports multi-GPU distributed training, the dataset size and model complexity make single-GPU training sufficiently efficient for this experimental evaluation.

4.2 Model Configuration

4.2.1 BIOT Tokenization Parameters

The biosignal tokenization module employs STFT to convert temporal signals into frequency-domain token representations. The STFT configuration determines the frequency and temporal resolution of the resulting tokens.

STFT Configuration:

- **FFT Size (`n_fft`):** 200 samples
- **Hop Length (`hop_length`):** 100 samples
- **Window Function:** Hann window
- **Sampling Rate:** 200 Hz (after resampling)

Resolution Calculations:

Frequency resolution:

$$\Delta f = \frac{f_s}{N} = \frac{200 \text{ Hz}}{200} = 1 \text{ Hz} \quad (4.1)$$

Temporal resolution (hop time):

$$\Delta t = \frac{H}{f_s} = \frac{100}{200 \text{ Hz}} = 0.5 \text{ seconds} \quad (4.2)$$

Frame overlap:

$$\text{Overlap} = \frac{N - H}{N} = \frac{200 - 100}{200} = 50\% \quad (4.3)$$

This configuration provides 1 Hz frequency resolution spanning 0-100 Hz (Nyquist limit) with 0.5-second temporal resolution and 50% overlap between consecutive frames. The

50% overlap ensures smooth temporal continuity while maintaining computational efficiency.

Token Sequence Length: The tokenization process transforms raw temporal samples into frequency-domain tokens through STFT. For a signal window of duration T seconds sampled at 200 Hz, the input contains $L = T \times 200$ time-domain samples. The STFT with `center=False` generates:

$$N_{\text{tokens}} = \left\lfloor \frac{L - N}{H} \right\rfloor + 1 = \left\lfloor \frac{T \cdot 200 - 200}{100} \right\rfloor + 1 = 2T - 1 \quad (4.4)$$

Concrete Examples:

- **2-2 configuration (5s total):** 1000 samples $\rightarrow$ 9 tokens/channel $\rightarrow$ 144 total tokens
- **3-3 configuration (7s total):** 1400 samples $\rightarrow$ 13 tokens/channel $\rightarrow$ 208 total tokens
- **4-4 configuration (9s total):** 1800 samples $\rightarrow$ 17 tokens/channel $\rightarrow$ 272 total tokens

Each token represents a 0.5-second window of frequency content across the 0-100 Hz range, providing a compact sequence representation suitable for transformer processing.

4.2.2 Baseline Transformer Configuration

The baseline BIOT architecture employs a Linear Attention Transformer as the sequence encoder, serving as the reference configuration against which xLSTM variants are compared [20].

Linear Attention Mechanism: The baseline uses linear attention rather than standard softmax attention, reducing computational complexity from quadratic to linear in sequence length. This enables efficient processing of longer EEG sequences while maintaining the Transformer’s ability to model long-range dependencies. The linear attention approximation computes attention scores without the expensive softmax operation, trading some representational capacity for computational efficiency.

Architecture Parameters:

- **Embedding Dimension:** 256 (matching xLSTM variants for fair comparison)
- **Num Blocks:** 8 (stacked transformer layers)
- **Attention Heads:** 8 (for multi-head attention mechanism)
- **Context Length:** 1024 tokens (maximum sequence length)
- **Dropout:** 0.2

Layer Composition: Each transformer block consists of a linear multi-head attention layer followed by a position-wise feedforward network, with residual connections and layer normalization applied around both sub-layers. This follows the standard Transformer architecture pattern but substitutes linear attention for computational efficiency.

For complete implementation details and hyperparameter specifications, refer to the original BIOT work [20] or the official implementation at <https://github.com/ycq091044/BIOT>.

4.2.3 xLSTM Component Configuration

mLSTM Configuration

The mLSTM components are configured with the following parameters:

- **Embedding Dimension:** 256
- **Context Length:** 1024 tokens (maximum sequence length)
- **Num Blocks:** 8 (stacked mLSTM layers)
- **Dropout:** 0.2 (applied within mLSTM cells)

The dropout rate of 0.2 provides regularization while maintaining the matrix memory’s key-value association capabilities. The mLSTM does not include feedforward networks, relying solely on the matrix memory mechanism for feature transformation.

sLSTM Configuration

The sLSTM components employ the following configuration:

- **Embedding Dimension:** 256
- **Context Length:** 1024 tokens (maximum sequence length)
- **Num Blocks:** 8 (stacked sLSTM layers)
- **Num Heads:** 8 (for multi-head memory operations)
- **Dropout:** 0.2 (applied within sLSTM cells)
- **Feedforward:** Enabled with gated activation
- **Projection Factor:** 1.3 (feedforward expansion ratio)
- **Activation Function:** GELU
- **Bias:** False

Combined Architecture Configuration

The combined sLSTM-mLSTM architecture employs an eight-block stack with the **xLSTM[7:1]** configuration:

- **Total Blocks:** 8
- **Block Composition:** 7 mLSTM blocks + 1 sLSTM block
- **sLSTM Position:** Block index 1 (mLSTM → sLSTM → mLSTM ($\times 6$))
- **Dropout:** 0.2 (applied within each cell)
- **Embedding Dimension:** 256 (consistent across all blocks)
- **Context Length:** 1024 tokens (maximum sequence length)

This configuration leverages the complementary strengths of both memory mechanisms: mLSTM’s parallelizable matrix memory for efficient computation and sLSTM’s scalar memory mixing for sequential pattern capture. The single sLSTM block positioned early in the stack (block 1) provides crucial memory mixing capabilities while maintaining computational efficiency through predominantly parallel mLSTM processing. The 7:1 ratio was identified as the best-performing mLSTM-to-sLSTM combination through systematic ablation studies in the original xLSTM work [2], where replacing some sLSTM blocks with mLSTM blocks yielded optimal language modeling performance.

4.2.4 Model Parameter Counts

The four architecture variants exhibit different parameter counts due to their distinct memory mechanisms and computational requirements:

Architecture	Parameters	Size (MB)
Baseline (Linear Transformer)	3,155,968	12.751
mLSTM-only	3,324,224	13.424
sLSTM-only	2,908,416	11.761
Combined (xLSTM[7:1])	3,272,248	13.216

Table 4.1: Model parameter counts and memory footprint for each architecture variant. All models have comparable complexity, with parameter counts ranging from 2.9M to 3.3M.

The mLSTM-only configuration has the highest parameter count (3.32M) due to the matrix memory structures in all eight blocks, which require storing key-value matrices. The sLSTM-only variant has the fewest parameters (2.91M) as scalar memory mechanisms and gated feedforward networks are more parameter-efficient than matrix operations. The combined xLSTM[7:1] architecture (3.27M parameters) sits between the two pure variants, reflecting its composition of seven mLSTM blocks and one sLSTM block. The baseline linear Transformer (3.16M parameters) provides a comparable parameter budget for fair comparison.

All models maintain similar complexity levels, ensuring that performance differences reflect architectural advantages rather than capacity disparities. The parameter counts are computed using PyTorch Lightning’s model summary functionality, which accounts for all trainable parameters including embeddings, attention mechanisms, memory cells, and classification heads.

4.2.5 Classification Head Configuration

The classification head processes the aggregated sequence representation through mean pooling across all tokens, followed by an ELU activation function. The ELU activation provides smooth gradients for negative inputs while maintaining identity for positive inputs, facilitating stable optimization. The head outputs a 6-dimensional vector corresponding to the number of event classes in the TUEV dataset. Training employs cross-entropy loss to optimize the model’s classification performance.

4.3 Training Configuration

4.3.1 Optimization Parameters

Optimizer Configuration: The AdamW optimizer combines adaptive learning rates with decoupled weight decay regularization, providing robust optimization for deep neural networks. The learning rate is set to $\alpha = 1 \times 10^{-3}$, with beta parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The epsilon parameter is configured as $\epsilon = 1 \times 10^{-8}$ for numerical stability. A weight decay coefficient of $\lambda = 1 \times 10^{-5}$ provides L2 regularization to prevent overfitting.

Batch Configuration: Training uses a batch size of 128 samples, which balances gradient estimation quality with memory efficiency and training speed. Gradients are applied directly without accumulation. Data loading employs 16 parallel workers with persistent worker mode enabled, meaning worker processes remain alive between epochs to reduce data loading overhead.

Training Duration: Each configuration is trained for a maximum of 100 epochs without early stopping. Instead, the dual checkpointing strategy (described in Section 4.3.3) captures the best models based on different metrics.

4.3.2 Data Loading Configuration

Dataset Split Parameters: The training data is split at the subject level, with 10% of training subjects allocated to the validation set. This subject-level splitting ensures that all samples from the same subject remain in the same partition, preventing data leakage and providing a more realistic evaluation of generalization performance. The random seed for splitting is configuration-specific, using values of 10, 42, 123, 456 or 999 depending on the experimental run. Training data is shuffled during loading to prevent batch correlation effects, while validation data maintains its original ordering for consistent evaluation.

Data Augmentation: No data augmentation is applied, as the experiments focus on evaluating base model performance without additional transformations. Signal normalization is performed during preprocessing using per-channel 95th percentile scaling, as described in Section 3.3.

4.3.3 Checkpointing Configuration

The training process employs a dual checkpointing strategy to capture models that excel according to different criteria. The first checkpoint strategy monitors validation cross-entropy loss, saving the model whenever the loss decreases to a new minimum. The second checkpoint strategy tracks validation balanced accuracy, preserving the model whenever accuracy increases to a new maximum.

Both checkpoints include complete model state, optimizer state, epoch number, and training configuration for reproducibility.

These settings ensure reproducibility across runs with the same seed, though slight variations may occur due to hardware-specific optimizations.

4.4 Experimental Execution

4.4.1 Parameter Sweep Implementation

The parameter sweep is implemented through an automated script (`run_sweep.py`) that systematically generates and executes all experimental configurations in the design space.

Experimental Factors: The parameter sweep covers three primary factors:

- **Model Architecture:** Four variants (Baseline, sLSTM-only, mLSTM-only, Combined)
- **Temporal Window:** Three configurations (2-2, 3-3, 4-4 seconds before/after events)

- **Random Seeds:** Five seeds (10, 42, 123, 456, 999) for statistical robustness

Temporal Window Rationale: The temporal window configurations are designed to explore the impact of contextual information surrounding detected events. Each configuration specifies the number of seconds before and after the central event annotation. For example, a 2-2 configuration creates a 5-second sample: 2 seconds before + 1 second event + 2 seconds after.

The choice of configurations is motivated by both theoretical and practical considerations:

- **Baseline Configuration (2-2):** This 5-second window matches the default configuration used in the original BIOT work [20], where TUEV samples are processed with symmetric 2-second contexts around events. Starting from this established baseline enables direct comparison with prior work.
- **Extended Context (3-3):** The 7-second window explores whether additional temporal context improves event characterization by capturing longer-range patterns or anticipatory signals.
- **Maximum Context (4-4):** The 9-second window tests the upper limit of useful temporal context. Further expansion beyond this range was not feasible due to Graphics Processing Unit (GPU) memory constraints with the available hardware (RTX 3090 with 24GB).

The symmetric temporal windows (`secondsBefore` = `secondsAfter`) maintain consistent pre-event and post-event context, simplifying interpretation and enabling fair comparison. This range spans 5-9 seconds total duration, covering clinically plausible contexts while respecting computational constraints.

The multi-seed evaluation addresses the stochastic nature of neural network training. Performance can vary significantly across random initializations due to different local optima, gradient noise, and data shuffling. By evaluating each configuration with five different seeds, the experimental design enables:

- Estimation of performance variability
- Calculation of confidence intervals
- Statistical significance testing between configurations
- Identification of robust vs. fragile architectural choices

Experimental Matrix: The full factorial design results in $4 \times 5 \times 3 = 60$ training runs, with each run producing two checkpointed models (loss-based and accuracy-based). This comprehensive coverage enables robust statistical analysis and clear identification of optimal configurations for different performance criteria.

Chapter 5

Results and Analysis

This chapter presents the experimental results from evaluating xLSTM variants in the BIOT architecture for EEG event classification using the TUEV dataset. The experimental design systematically compares four architectural variants (sLSTM, mLSTM, combined mLSTM+sLSTM, and a Linear Transformer baseline) across three temporal window configurations (5 s, 7 s, 9 s). Each configuration was run five times with different random seeds (10, 42, 123, 456, 999) to ensure statistical robustness.

5.0.1 Model Selection Criterion

All results use checkpoints selected by balanced accuracy, not validation loss. The TUEV dataset exhibits severe class imbalance (66.77% majority class), causing validation loss to plateau and fluctuate after initial convergence. Models can minimize cross-entropy loss by biasing toward the majority class while achieving poor balanced accuracy. Since balanced accuracy equally weights all classes, it is the appropriate selection criterion for this imbalanced multi-class task. All reported metrics, confusion matrices, and statistical comparisons use balanced-accuracy checkpoints exclusively.

5.1 Overall Performance Comparison

We analyze four key metrics (see Section 3.4.3 for definitions): balanced accuracy (robust to class imbalance), Cohen’s κ (agreement beyond chance), weighted F1 (precision–recall harmonic mean weighted by support), and Area Under the Precision-Recall Curve (AUCPR) Macro (ranking quality for rare events). Overall accuracy is not emphasized because it is misleading under imbalance; when shown, it serves only for completeness in statistical tests.

5.1.1 Complete Performance Results

Table 5.1 summarizes performance across all 60 runs (4 architectures $\times$ 3 windows $\times$ 5 seeds). Values are mean $\pm$ standard deviation over the five seeds; bold indicates the best score within a given window length.

5 s windows. sLSTM attains the best mean balanced accuracy ($52.15\% \pm 1.37\%$) with very low variability, substantially more stable than the Linear Transformer ($52.06\% \pm 4.06\%$). The combined model is stable ($50.03\% \pm 1.72\%$) but trails pure sLSTM. mLSTM is intermediate ($50.93\% \pm 3.14\%$).

Table 5.1: Performance comparison across architectures and temporal window configurations. Values are mean $\pm$ std over five seeds using balanced-accuracy checkpointing. Boldface indicates the best value per window.

Architecture	Win.	Bal. Acc.	Cohen κ	F1 (W)	AUCPR (Macro)
Linear Transf.	5s	0.521 $\pm$ 0.041	0.457 $\pm$ 0.049	0.712 $\pm$ 0.034	0.468 $\pm$ 0.013
Linear Transf.	7s	0.501 $\pm$ 0.043	0.465 $\pm$ 0.030	0.717 $\pm$ 0.022	0.465 $\pm$ 0.039
Linear Transf.	9s	0.485 $\pm$ 0.037	0.425 $\pm$ 0.048	0.691 $\pm$ 0.031	0.453 $\pm$ 0.026
mLSTM	5s	0.509 $\pm$ 0.031	0.439 $\pm$ 0.037	0.707 $\pm$ 0.017	0.487 $\pm$ 0.030
mLSTM	7s	0.507 $\pm$ 0.029	0.474 $\pm$ 0.012	0.727 $\pm$ 0.006	0.484 $\pm$ 0.016
mLSTM	9s	0.464 $\pm$ 0.066	0.422 $\pm$ 0.063	0.702 $\pm$ 0.029	0.459 $\pm$ 0.036
sLSTM	5s	0.522 $\pm$ 0.014	0.471 $\pm$ 0.044	0.720 $\pm$ 0.027	0.513 $\pm$ 0.016
sLSTM	7s	0.485 $\pm$ 0.055	0.466 $\pm$ 0.091	0.720 $\pm$ 0.047	0.493 $\pm$ 0.042
sLSTM	9s	0.525 $\pm$ 0.038	0.479 $\pm$ 0.063	0.726 $\pm$ 0.037	0.486 $\pm$ 0.039
mLSTM+sLSTM	5s	0.500 $\pm$ 0.017	0.440 $\pm$ 0.050	0.705 $\pm$ 0.029	0.489 $\pm$ 0.013
mLSTM+sLSTM	7s	0.505 $\pm$ 0.028	0.443 $\pm$ 0.026	0.707 $\pm$ 0.016	0.479 $\pm$ 0.020
mLSTM+sLSTM	9s	0.501 $\pm$ 0.011	0.431 $\pm$ 0.045	0.701 $\pm$ 0.027	0.472 $\pm$ 0.023

7 s windows. mLSTM performs best (50.66% $\pm$ 2.87%) and is the most stable in this regime. The Linear Transformer remains competitive (50.08% $\pm$ 4.29%), while sLSTM dips (48.45% $\pm$ 5.50%). The combined model is again stable and near the middle (50.46% $\pm$ 2.82%).

9 s windows. sLSTM rebounds to the top (52.46% $\pm$ 3.81%); the combined model is remarkably stable (50.10% $\pm$ 1.14%). The Linear Transformer decreases monotonically with context length (48.45% $\pm$ 3.73%). mLSTM degrades substantially (46.40% $\pm$ 6.59%).

5.1.2 Architectural Comparison (Across Windows)

The following architectural characterizations are based on five runs per configuration. While clear trends emerge, the observed patterns may partly reflect sampling variability; additional runs would strengthen confidence in distinguishing systematic architectural behaviors from random variation.

sLSTM. Strong overall with a U-shaped window-length response: best at 5 s and 9 s, pronounced dip at 7 s. This suggests sensitivity of scalar-memory dynamics to sequence length.

mLSTM. Inverse trend to sLSTM: best at 7 s, degrades at 9 s. Matrix memory appears less suited to long windows in this task.

Combined mLSTM+sLSTM. Consistently stable (especially at 9 s) but never best-in-class. The fixed 7:1 ratio likely dilutes sLSTM benefits while not fully realizing mLSTM’s strengths.

Linear Transformer. Competitive at short windows but declines with longer context. Variance is highest at 5 s (CV 7.8%), indicating less stable optimization than xLSTM variants.

For readability, the main figure focuses on AUCPR Macro. In Figure 5.1, higher values are better. The median line shows the typical result across seeds, and the spread of the boxes shows how stable training is.

At 5s, sLSTM has a clearly higher median AUCPR than the Linear Transformer and the distributions overlap less. This matches the significant result at 5s reported in Table 5.2. At 7s and 9s, the boxplots overlap more strongly across architectures, so improvements are less consistent and often within seed variance. The combined model is usually stable (tight boxes) but does not reach the best median values.

The corresponding plots for balanced accuracy, Cohen’s κ , and weighted F1 are provided in Appendix B.1.

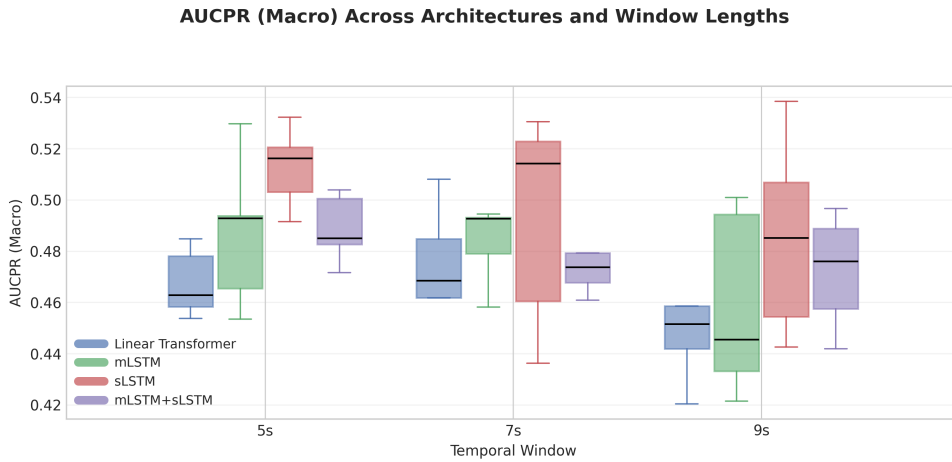


Figure 5.1: AUCPR Macro across architectures and temporal window configurations (5s, 7s, 9s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture. At 5s, sLSTM shows a higher median than the Linear Transformer with less overlap.

5.2 Statistical Significance Testing

We perform pairwise statistical comparisons between the Linear Transformer baseline and the best-performing xLSTM variant at each window length using paired t-tests and Wilcoxon signed-rank tests across seeds ($\alpha = 0.05$). At 5s and 9s windows, we compare against sLSTM (the top performer), while at 7s windows we compare against mLSTM (the best-performing architecture at this window length). As described in Section 3.5.2, the statistical tests are performed on the individual run results (5 seeds per architecture), while the tables below report the mean values for readability. Reported p-values are uncorrected; they should be interpreted alongside effect sizes. Given the limited sample size ($n=5$ seeds), statistical significance should be interpreted cautiously; we emphasize trends and consistency across seeds over individual p-values.

5.2.1 5-Second Windows

Summary. At 5s, sLSTM significantly improves ranking quality (AUCPR Macro) without a corresponding balanced-accuracy gain.

Table 5.2: Linear Transformer vs. sLSTM at 5 s (balanced-accuracy checkpoints). Positive differences favor sLSTM.

Metric	Lin. Transf.	sLSTM	t-test p	Wilcoxon p	Interpretation
Balanced Accuracy	0.521	0.522	0.974	0.812	Equivalent
Cohen’s κ	0.457	0.471	0.583	0.438	Higher, not significant
F1 (Weighted)	0.712	0.720	0.651	0.438	Higher, not significant
Accuracy	0.689	0.711	0.252	0.312	Higher, not significant
AUCPR (Macro)	0.468	0.513	0.005	0.062	Significant (t-test)

5.2.2 7-Second Windows

Table 5.3: Linear Transformer vs. mLSTM at 7 s (balanced-accuracy checkpoints). Positive differences favor mLSTM.

Metric	Lin. Transf.	mLSTM	t-test p	Wilcoxon p	Interpretation
Balanced Accuracy	0.501	0.507	0.749	0.812	Equivalent
Cohen’s κ	0.465	0.474	0.394	0.438	Higher, not significant
F1 (Weighted)	0.717	0.727	0.298	0.312	Higher, not significant
Accuracy	0.689	0.711	0.215	0.188	Higher, not significant
AUCPR (Macro)	0.465	0.484	0.315	0.438	Higher, not significant

Summary. No significant differences at 7 s between Linear Transformer and mLSTM; all metrics are within experimental variance despite mLSTM’s numerically higher performance.

5.2.3 9-Second Windows

Table 5.4: Linear Transformer vs. sLSTM at 9 s (balanced-accuracy checkpoints). Positive differences favor sLSTM.

Metric	Lin. Transf.	sLSTM	t-test p	Wilcoxon p	Interpretation
Balanced Accuracy	0.485	0.525	0.119	0.188	Higher, not significant
Cohen’s κ	0.425	0.479	0.055	0.062	Marginal
F1 (Weighted)	0.691	0.726	0.073	0.062	Marginal
Accuracy	0.667	0.714	0.029	0.062	Significant (t-test)
AUCPR (Macro)	0.453	0.486	0.115	0.062	Higher, not significant

Summary. At 9 s, sLSTM yields a significant accuracy gain and consistent (though marginal) advantages in κ and weighted F1; balanced-accuracy improvement is larger in magnitude than at 5–7 s but not significant.

5.2.4 Interpretation

Window-dependent benefits. sLSTM advantages are window-specific: AUCPR gains at 5 s and accuracy gains at 9 s, but underperforms at 7 s where mLSTM achieves the best results (though without significant advantages over the Linear Transformer baseline).

Metric-specific effects. Improvements manifest differently by window: sLSTM provides ranking quality gains at short context and decision accuracy gains at long context.

Balanced accuracy—our selection criterion—never reaches significance, indicating architectural changes mainly influence confidence calibration and error distribution rather than class-balanced decision boundaries.

Practical significance. Effect sizes are modest ($< 5\%$ absolute). Given xLSTM variants’ higher compute cost and the lack of consistent advantages across window lengths, deployment benefits are constrained; see Section 5.3.

5.3 Computational Efficiency Analysis

Training time is a key practical constraint. Table 5.5 reports the mean wall-clock time per complete run (**100 epochs**) under identical hardware and batch sizes.

Table 5.5: Training runtime per complete run (100 epochs). Values are mean time with standard deviation in minutes over five seeds.

Architecture	Window (s)	N Runs	Mean Runtime	Std (min)
Linear Transformer	5	5	1h 49m	2.7
Linear Transformer	7	5	2h 38m	1.0
Linear Transformer	9	5	3h 15m	0.6
mLSTM	5	5	5h 30m	9.8
mLSTM	7	5	7h 47m	5.3
mLSTM	9	5	10h 57m	20.7
mLSTM+sLSTM	5	5	5h 32m	11.9
mLSTM+sLSTM	7	5	7h 37m	9.4
mLSTM+sLSTM	9	5	10h 27m	9.6
sLSTM	5	5	4h 10m	10.1
sLSTM	7	5	5h 31m	4.3
sLSTM	9	5	6h 40m	6.7

Linear Transformer. Fastest across all settings (1h49m–3h15m), delivering $2.3\text{--}3.4\times$ speedups over xLSTM. Very low variance indicates predictable compute. Runtime scales near-linearly with window length.

sLSTM. Most efficient xLSTM (4h10m–6h40m) and the best performance–efficiency compromise. Variance is moderate; scaling with window length is sublinear.

mLSTM. Most expensive (5h30m–10h57m) with superlinear scaling and high variance at 9 s, implying less predictable requirements as context grows.

Combined. Nearly identical to mLSTM in cost, confirming that the 7:1 mixture is dominated by mLSTM blocks. Variance is somewhat more controlled than pure mLSTM at 9 s.

5.4 Class-Specific Performance Analysis

While aggregate metrics provide overall performance assessment, understanding class-level behavior is essential for evaluating model utility in clinical settings where minority

classes often represent critical pathological events. We analyze normalized confusion matrices aggregated across all five random seeds, where diagonal entries represent per-class recall (sensitivity) and off-diagonal entries reveal systematic misclassification patterns. Values are reported as mean $\pm$ standard deviation to capture both typical performance and cross-seed stability.

5.4.1 Per-Class Recall Patterns

Baseline Performance at 5-Second Windows

Table 5.6: Normalized confusion matrix for Linear Transformer at 5 s windows, averaged over five seeds. Diagonal entries (bold) show per-class recall; off-diagonal entries show misclassification patterns.

True\Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.082 $\pm$ 0.164	0.259 $\pm$ 0.135	0.140 $\pm$ 0.109	0.070 $\pm$ 0.067	0.123 $\pm$ 0.150	0.325 $\pm$ 0.082
Class 1	0.027 $\pm$ 0.027	0.411 $\pm$ 0.119	0.201 $\pm$ 0.035	0.010 $\pm$ 0.009	0.229 $\pm$ 0.114	0.122 $\pm$ 0.061
Class 2	0.047 $\pm$ 0.022	0.144 $\pm$ 0.038	0.567 $\pm$ 0.064	0.065 $\pm$ 0.041	0.063 $\pm$ 0.035	0.113 $\pm$ 0.037
Class 3	0.005 $\pm$ 0.003	0.002 $\pm$ 0.005	0.022 $\pm$ 0.019	0.664 $\pm$ 0.046	0.086 $\pm$ 0.045	0.221 $\pm$ 0.034
Class 4	0.007 $\pm$ 0.006	0.006 $\pm$ 0.005	0.029 $\pm$ 0.029	0.038 $\pm$ 0.020	0.605 $\pm$ 0.099	0.316 $\pm$ 0.088
Class 5	0.029 $\pm$ 0.018	0.011 $\pm$ 0.005	0.047 $\pm$ 0.011	0.029 $\pm$ 0.014	0.089 $\pm$ 0.034	0.794 $\pm$ 0.030

The baseline reveals severe performance stratification by class frequency. Class 0 (1.93% of training samples) achieves only 8.2% mean recall with standard deviation (16.4%) exceeding the mean, indicating highly unstable predictions dominated by random seed effects. The majority of Class 0 instances are misclassified as Class 5 (32.5%) or Class 1 (25.9%), suggesting the model defaults to more frequent classes when confronted with rare patterns.

Moderate-frequency classes show intermediate performance: Class 1 (41.1% recall), Class 2 (56.7%), Class 3 (66.4%), and Class 4 (60.5%). Class 3 and Class 4 exhibit substantial confusion with Class 5 (22.1% and 31.6% respectively), reflecting the challenge of discriminating artifact patterns from background activity. The majority class (Class 5, 66.77% of samples) achieves 79.4% recall with low variance (3.0%), demonstrating stable high performance on frequent patterns.

xLSTM Performance at Optimal Configurations

Comparing sLSTM to the baseline reveals minimal architectural impact on class-specific discrimination. Class 0 recall actually decreases (5.1% vs. 8.2%), though neither value represents clinically useful performance. The primary difference emerges in the majority class: sLSTM achieves 82.6% recall on Class 5 with substantially lower variance (1.5% vs. 3.0%), consistent with its superior overall optimization stability. Intermediate classes show nearly identical recall patterns, with differences well within cross-seed variance.

Performance at 7-Second Windows

At 7 s windows, architectural differences become more pronounced. Table 5.8 shows the Linear Transformer baseline at this window length.

Table 5.7: Normalized confusion matrix for sLSTM at 5 s windows, averaged over five seeds.

True\Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.051 $\pm$ 0.078	0.400 $\pm$ 0.058	0.238 $\pm$ 0.022	0.056 $\pm$ 0.023	0.062 $\pm$ 0.059	0.194 $\pm$ 0.085
Class 1	0.027 $\pm$ 0.018	0.420 $\pm$ 0.143	0.184 $\pm$ 0.015	0.007 $\pm$ 0.005	0.108 $\pm$ 0.063	0.255 $\pm$ 0.090
Class 2	0.093 $\pm$ 0.060	0.116 $\pm$ 0.062	0.525 $\pm$ 0.095	0.047 $\pm$ 0.044	0.077 $\pm$ 0.025	0.142 $\pm$ 0.052
Class 3	0.004 $\pm$ 0.008	0.000 $\pm$ 0.000	0.012 $\pm$ 0.019	0.664 $\pm$ 0.120	0.097 $\pm$ 0.042	0.224 $\pm$ 0.086
Class 4	0.001 $\pm$ 0.001	0.010 $\pm$ 0.007	0.011 $\pm$ 0.006	0.030 $\pm$ 0.016	0.643 $\pm$ 0.057	0.305 $\pm$ 0.053
Class 5	0.017 $\pm$ 0.018	0.018 $\pm$ 0.007	0.030 $\pm$ 0.013	0.011 $\pm$ 0.006	0.097 $\pm$ 0.020	0.826 $\pm$ 0.015

Table 5.8: Normalized confusion matrix for Linear Transformer at 7 s windows, averaged over five seeds.

True\Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.066 $\pm$ 0.089	0.383 $\pm$ 0.130	0.086 $\pm$ 0.039	0.090 $\pm$ 0.043	0.141 $\pm$ 0.155	0.234 $\pm$ 0.036
Class 1	0.028 $\pm$ 0.028	0.485 $\pm$ 0.117	0.191 $\pm$ 0.070	0.008 $\pm$ 0.009	0.235 $\pm$ 0.114	0.053 $\pm$ 0.017
Class 2	0.079 $\pm$ 0.071	0.181 $\pm$ 0.096	0.466 $\pm$ 0.100	0.088 $\pm$ 0.049	0.065 $\pm$ 0.016	0.119 $\pm$ 0.061
Class 3	0.002 $\pm$ 0.004	0.009 $\pm$ 0.009	0.016 $\pm$ 0.013	0.593 $\pm$ 0.082	0.104 $\pm$ 0.051	0.275 $\pm$ 0.061
Class 4	0.002 $\pm$ 0.003	0.011 $\pm$ 0.009	0.013 $\pm$ 0.011	0.071 $\pm$ 0.028	0.606 $\pm$ 0.073	0.296 $\pm$ 0.092
Class 5	0.022 $\pm$ 0.009	0.015 $\pm$ 0.014	0.034 $\pm$ 0.010	0.033 $\pm$ 0.017	0.107 $\pm$ 0.037	0.788 $\pm$ 0.066

The Linear Transformer at 7 s maintains relatively stable performance across most classes, with Class 1 achieving 48.5% recall, slightly higher than at 5 s. However, Class 0 performance remains poor at 6.6%, with high variance indicating continued instability on rare classes.

Table 5.9: Normalized confusion matrix for mLSTM at 7 s windows, averaged over five seeds. Best overall performance at this window length.

True\Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.083 $\pm$ 0.108	0.425 $\pm$ 0.115	0.044 $\pm$ 0.031	0.139 $\pm$ 0.061	0.019 $\pm$ 0.022	0.289 $\pm$ 0.053
Class 1	0.030 $\pm$ 0.028	0.532 $\pm$ 0.091	0.141 $\pm$ 0.037	0.010 $\pm$ 0.006	0.114 $\pm$ 0.078	0.171 $\pm$ 0.112
Class 2	0.076 $\pm$ 0.053	0.137 $\pm$ 0.030	0.483 $\pm$ 0.087	0.053 $\pm$ 0.028	0.100 $\pm$ 0.026	0.150 $\pm$ 0.043
Class 3	0.002 $\pm$ 0.005	0.007 $\pm$ 0.009	0.016 $\pm$ 0.016	0.579 $\pm$ 0.036	0.160 $\pm$ 0.021	0.235 $\pm$ 0.052
Class 4	0.005 $\pm$ 0.005	0.011 $\pm$ 0.010	0.012 $\pm$ 0.012	0.008 $\pm$ 0.009	0.546 $\pm$ 0.077	0.417 $\pm$ 0.064
Class 5	0.028 $\pm$ 0.017	0.024 $\pm$ 0.013	0.020 $\pm$ 0.007	0.006 $\pm$ 0.003	0.106 $\pm$ 0.010	0.815 $\pm$ 0.020

mLSTM at 7 s achieves the highest Class 1 recall across all configurations (53.2%), suggesting matrix memory better captures patterns in this moderately frequent class at intermediate sequence lengths. However, Class 0 remains problematic (8.3%), and Class 4 recall drops to 54.6% with increased confusion with Class 5 (41.7%), the highest misclassification rate observed for this class pair. This trade-off pattern—improved performance on some classes at the expense of others—explains why mLSTM’s aggregate balanced accuracy remains competitive without clearly dominating.

Performance at 9-Second Windows

At 9 s windows, the performance landscape shifts again. Table 5.10 shows the Linear Transformer baseline at this extended window length.

Table 5.10: Normalized confusion matrix for Linear Transformer at 9 s windows, averaged over five seeds.

True\Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.017 $\pm$ 0.028	0.349 $\pm$ 0.121	0.240 $\pm$ 0.069	0.078 $\pm$ 0.061	0.031 $\pm$ 0.048	0.286 $\pm$ 0.049
Class 1	0.007 $\pm$ 0.005	0.404 $\pm$ 0.183	0.270 $\pm$ 0.076	0.008 $\pm$ 0.008	0.175 $\pm$ 0.102	0.135 $\pm$ 0.076
Class 2	0.041 $\pm$ 0.024	0.162 $\pm$ 0.075	0.512 $\pm$ 0.117	0.099 $\pm$ 0.060	0.053 $\pm$ 0.040	0.134 $\pm$ 0.068
Class 3	0.009 $\pm$ 0.012	0.007 $\pm$ 0.010	0.019 $\pm$ 0.010	0.616 $\pm$ 0.153	0.068 $\pm$ 0.048	0.281 $\pm$ 0.130
Class 4	0.008 $\pm$ 0.005	0.008 $\pm$ 0.012	0.022 $\pm$ 0.012	0.061 $\pm$ 0.041	0.584 $\pm$ 0.054	0.317 $\pm$ 0.052
Class 5	0.031 $\pm$ 0.019	0.012 $\pm$ 0.009	0.049 $\pm$ 0.018	0.036 $\pm$ 0.028	0.099 $\pm$ 0.020	0.774 $\pm$ 0.044

The Linear Transformer at 9 s shows continued degradation in Class 0 (1.7% recall) and substantial decline in Class 5 (77.4%, down from 79.4% at 5 s and 78.8% at 7 s). Class 1 recall also drops to 40.4%, indicating that extended context length degrades the baseline’s ability to maintain performance across classes. This monotonic decline with window length suggests attention mechanisms struggle to effectively leverage longer temporal contexts in this task.

Table 5.11: Normalized confusion matrix for sLSTM at 9 s windows, averaged over five seeds. Best overall balanced accuracy at this window length.

True\Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.142 $\pm$ 0.094	0.371 $\pm$ 0.144	0.223 $\pm$ 0.151	0.096 $\pm$ 0.061	0.014 $\pm$ 0.022	0.153 $\pm$ 0.075
Class 1	0.047 $\pm$ 0.058	0.444 $\pm$ 0.173	0.238 $\pm$ 0.118	0.007 $\pm$ 0.010	0.078 $\pm$ 0.041	0.185 $\pm$ 0.153
Class 2	0.047 $\pm$ 0.030	0.170 $\pm$ 0.015	0.516 $\pm$ 0.089	0.059 $\pm$ 0.043	0.061 $\pm$ 0.041	0.146 $\pm$ 0.071
Class 3	0.020 $\pm$ 0.038	0.002 $\pm$ 0.004	0.006 $\pm$ 0.011	0.638 $\pm$ 0.170	0.075 $\pm$ 0.080	0.258 $\pm$ 0.106
Class 4	0.011 $\pm$ 0.008	0.015 $\pm$ 0.009	0.015 $\pm$ 0.011	0.038 $\pm$ 0.039	0.576 $\pm$ 0.055	0.345 $\pm$ 0.048
Class 5	0.022 $\pm$ 0.010	0.029 $\pm$ 0.006	0.027 $\pm$ 0.017	0.013 $\pm$ 0.014	0.078 $\pm$ 0.036	0.831 $\pm$ 0.045

At 9 s windows, sLSTM achieves 14.2% recall on Class 0, the highest observed across all configurations. However, the high standard deviation (9.4%, representing 66% of the mean) indicates this improvement is driven by occasional favorable seeds rather than systematic architectural advantage. Class 5 recall remains strong (83.1%), and intermediate classes show performance comparable to 5 s configurations. The recovery from the 7 s performance dip suggests sLSTM’s scalar memory mechanisms operate more effectively at both short and extended sequence lengths, with a performance valley at intermediate scales.

5.4.2 Statistical Comparison of Architectural Effects

To rigorously assess whether observed class-level differences reflect genuine architectural advantages or random variation, we perform paired statistical tests comparing Linear Transformer and sLSTM at each window length.

5-Second Windows

Table 5.12: Statistical comparison of per-class recall: Linear Transformer vs. sLSTM at 5 s windows. Tests are paired across seeds; $\alpha = 0.05$.

Class	Linear	sLSTM	t-stat	p (t-test)	p (Wilcoxon)
Class 0	0.082±0.164	0.051±0.078	0.33	0.756	1.000
Class 1	0.411±0.119	0.420±0.143	−0.13	0.902	0.812
Class 2	0.567±0.064	0.525±0.095	0.92	0.410	0.312
Class 3	0.664±0.046	0.664±0.120	0.01	0.989	1.000
Class 4	0.605±0.099	0.643±0.057	−0.71	0.515	0.812
Class 5	0.794±0.030	0.826±0.015	−2.38	0.076	0.062

At 5 s windows, no class-level differences reach statistical significance at $\alpha = 0.05$. Class 5 shows the strongest trend toward sLSTM advantage (recall difference: +3.2 percentage points, $p=0.076$), approaching but not achieving significance. This near-significant improvement in majority-class recall, combined with marginal differences in other classes, explains why sLSTM achieves significantly better AUCPR Macro (ranking quality; Section 5.2) without significant balanced accuracy improvement. The architectural benefit manifests primarily in confidence calibration for the majority class rather than improved discrimination of minority classes.

In summary, at 5 s windows, sLSTM provides no significant class-level improvements but shows trends toward better majority-class performance and ranking quality.

7-Second Windows

Table 5.13: Statistical comparison of per-class recall: Linear Transformer vs. mLSTM at 7 s windows.

Class	Linear	mLSTM	t-stat	p (t-test)	p (Wilcoxon)
Class 0	0.066±0.089	0.083±0.108	−0.29	0.789	0.625
Class 1	0.485±0.117	0.532±0.091	−1.05	0.354	0.438
Class 2	0.466±0.100	0.483±0.087	−0.32	0.763	0.812
Class 3	0.593±0.082	0.579±0.036	0.32	0.764	1.000
Class 4	0.606±0.073	0.546±0.077	0.96	0.390	0.812
Class 5	0.788±0.066	0.815±0.020	−0.80	0.469	0.625

At 7 s windows, no class-level differences approach significance (all $p>0.35$). mLSTM shows numerically higher recall for Classes 0, 1, 2, and 5, with the largest improvement in Class 1 (+4.7 percentage points, $p=0.354$). However, Class 4 recall decreases substantially (−6.0 percentage points, $p=0.390$), indicating a trade-off where matrix memory improves performance on some classes at the expense of others. This mixed pattern explains why mLSTM’s aggregate balanced accuracy remains competitive without clearly dominating (Section 5.3). The higher Class 1 recall (53.2%, the best across all configurations) suggests matrix memory mechanisms are particularly effective for this moderately frequent class at intermediate sequence lengths.

In summary, at 7 s windows, mLSTM achieves the best Class 1 recall across all configurations but shows trade-offs in Class 4, resulting in no significant aggregate advantages

over the Linear Transformer baseline.

9-Second Windows

Table 5.14: Statistical comparison of per-class recall: Linear Transformer vs. sLSTM at 9 s windows.

Class	Linear	sLSTM	t-stat	p (t-test)	p (Wilcoxon)
Class 0	0.017 $\pm$ 0.028	0.142 $\pm$ 0.094	-2.84	0.047*	0.125
Class 1	0.404 $\pm$ 0.183	0.444 $\pm$ 0.173	-0.42	0.698	0.812
Class 2	0.512 $\pm$ 0.117	0.516 $\pm$ 0.089	-0.07	0.948	1.000
Class 3	0.616 $\pm$ 0.153	0.638 $\pm$ 0.170	-0.22	0.839	1.000
Class 4	0.584 $\pm$ 0.054	0.576 $\pm$ 0.055	0.22	0.834	1.000
Class 5	0.774 $\pm$ 0.044	0.831 $\pm$ 0.045	-3.97	0.016*	0.062

At 9 s windows, sLSTM achieves statistically significant improvements in Class 0 ($p=0.047$; +12.5 percentage points) and Class 5 ($p=0.016$; +5.7 percentage points). The Class 0 improvement is substantial in relative terms ($8.3\times$ increase from 1.7% to 14.2%), representing the first instance where any architecture achieves double-digit recall on the rarest class. However, the clinical utility remains questionable: 14.2% recall means 85.8% of minority-class instances are still misclassified. The high standard deviation (9.4%) relative to the mean indicates this improvement is highly variable across seeds, with the Wilcoxon test ($p=0.125$) not reaching significance, suggesting potential sensitivity to outlier seeds.

The Class 5 improvement is more robust, with both parametric and non-parametric tests approaching or reaching significance. This majority-class advantage combines with marginal improvements across intermediate classes (Classes 1–3 show small non-significant increases) to produce the significant overall accuracy improvement observed in aggregate metrics (Section 5.4). Notably, Class 4 shows no improvement, maintaining the pattern where extended context benefits some but not all classes.

In summary, at 9 s windows, sLSTM achieves significant improvements in the rarest class (Class 0) and the majority class (Class 5), driving its overall performance recovery at extended context lengths.

5.5 Summary of Key Findings

5.5.1 Performance Characteristics

- **Best overall (balanced acc.).** sLSTM leads at 5 s ($52.15\% \pm 1.37\%$) and 9 s ($52.46\% \pm 3.81\%$), but drops at 7 s ($48.45\% \pm 5.50\%$), indicating sequence-length sensitivity.
- **Most consistent.** The combined mLSTM+sLSTM is highly stable across windows (50.03–50.46%), with the lowest variability at 9 s ($\pm 1.14\%$), but never achieves the top score.
- **Window-specific peak.** mLSTM peaks at 7 s ($50.66\% \pm 2.87\%$) and collapses at 9 s ($46.40\% \pm 6.59\%$).

- **Baseline competitiveness.** The Linear Transformer is competitive at 5–7 s while training 2.3–3.4× faster; it declines with longer context.

5.5.2 Statistical Significance

- **Window dependence.** sLSTM achieves significant AUCPR gains at 5 s ($p=0.005$) and significant accuracy gains at 9 s ($p=0.029$); no significant differences at 7 s.
- **Metric dependence.** Gains shift from ranking quality (short windows) to decision accuracy (long windows). Balanced accuracy never reaches significance.
- **Effect sizes.** All significant improvements are modest ($<5\%$ absolute), tempering practical impact.

5.5.3 Architectural Insights

- **Sequence-length sensitivity.** sLSTM and mLSTM prefer different operating ranges (U-shaped vs. inverted-U), pointing to distinct memory–timescale interactions.
- **Hybrid trade-offs.** A fixed 7:1 mLSTM:sLSTM mix prioritizes stability but mutes peak performance; alternative ratios or adaptive mixing merit exploration.
- **Compute scaling.** xLSTM variants incur 2–3× training cost over the baseline; mLSTM scales superlinearly with window length.

5.5.4 Practical Implications

- **Deployment choice.** For this task, sLSTM (5 s) offers the best accuracy–stability point, whereas the Linear Transformer (5 s) provides the fastest, most predictable training with competitive accuracy.
- **Data imbalance dominates.** Architectural changes and longer context alone do not fix minority-class recall; future work should prioritize class-imbalance remedies (e.g., cost-sensitive learning, reweighting, resampling, augmentation).

Chapter 6

Discussion

This chapter discusses the experimental results in the context of biosignal processing research. It examines the performance improvements observed with xLSTM variants and discusses what they mean for clinical applications. The discussion covers both the achievements and limitations of integrating xLSTM architectures within the BIOT framework.

6.1 Performance Interpretation and Window Effects

The experimental results show that performance depends on the window length used. There is no single architecture that performs best in all cases. sLSTM achieves best performance at 5 s ($52.15\% \pm 1.37\%$ balanced accuracy) and 9 s windows ($52.46\% \pm 3.81\%$), but performs worse at 7 s ($48.45\% \pm 5.50\%$). In contrast, mLSTM performs best at 7 s windows ($50.66\% \pm 2.87\%$) where sLSTM performs worst. The Linear Transformer baseline performs well at 5 s ($52.06\% \pm 4.06\%$) but gets worse with longer windows.

6.1.1 Window-Dependent Benefits

The results show that xLSTM does not consistently outperform the baseline. Instead, each memory mechanism works better with specific window lengths. At 5 s windows, the Linear Transformer achieves 52.06% balanced accuracy, which is almost the same as sLSTM's 52.15%. The Linear Transformer also trains $2.3\times$ faster. Only at 9 s windows does sLSTM show clear advantages (52.46% vs. 48.45% for Linear Transformer), though this comes with longer training time (60% slower) and high variance ($\pm 3.81\%$).

Several factors can explain these limited and window-dependent benefits:

Sequence-Length Mismatch: The BIOT tokenization creates relatively short sequences (9-17 tokens per channel for 5-9 s windows, 144-272 total tokens). These sequences may not need the advanced long-term memory capabilities that xLSTM provides. The enhanced memory structures designed for long-sequence language modeling may be too complex for biosignal sequences where patterns occur within shorter time spans.

Local Pattern Sufficiency: The good baseline performance suggests that local patterns captured by linear attention may be enough for seizure classification. Neurological events in the TUEV dataset may be characterized by local features (frequency patterns, amplitude changes, channel correlations) that do not need complex long-range dependency modeling.

Training Dynamics Dominance: The high variability across runs (coefficient of variation 6-10%) shows that training dynamics, random initialization, and optimization challenges have a bigger impact on performance than the architecture itself. This suggests

that improving training stability may give better results than using more complex architectures for this task.

Hyperparameter Suboptimality: All architectures use hyperparameters optimized for the Linear Transformer baseline (dropout 0.2, 8 blocks, embedding dimension 256). The xLSTM variants may not work well with these settings. The window-dependent patterns might show hyperparameter mismatch rather than true architectural limitations.

6.1.2 Architecture-Specific Window Behaviors

Each architecture responds differently to window length:

Linear Transformer: Shows consistent degradation with increasing window length (52.06% $\rightarrow$ 50.08% $\rightarrow$ 48.45%). This suggests fundamental limitations in linear attention for longer sequences, where attention is spread across more tokens.

sLSTM: Shows a U-shaped pattern with a strong drop at 7s windows. Class 2 recall drops from 52.5% (5s) to 42.9% (7s) with statistical significance ($p=0.043$). This window-specific failure may be caused by gradient flow issues at intermediate sequence lengths, memory capacity mismatch, or hyperparameter sensitivity. The recovery at 9s suggests scalar memory can adapt to longer sequences once they exceed a certain length.

mLSTM: Shows an inverted pattern, peaking at 7s where sLSTM fails but collapsing at 9s (46.40%). This suggests different optimal operating ranges. The poor performance at 9s may be caused by higher memory requirements, difficulty maintaining key-value associations, or parallelization limits with very long sequences.

Combined Architecture: The mLSTM+sLSTM architecture shows very stable performance across windows (50.03% to 50.46%) but never achieves the best performance. The 7:1 ratio may reduce the benefits of each mechanism instead of combining them effectively.

6.1.3 Domain Transfer Challenges

The modest improvements contrast with xLSTM’s large gains in natural language processing [2]. Language has rich long-range dependencies requiring information from hundreds or thousands of tokens. In contrast, the tokenized EEG sequences (9-17 tokens per channel) mainly encode local patterns. The BIOT tokenization [20] converts 0.5-second windows of frequency content into discrete tokens, creating sequences where relevant patterns may be captured within short time spans.

Additionally, biosignal tokens represent frequency-domain features from continuous signals without the discrete semantic structure of language [20]. Neurological events in the TUEV dataset [10] may have more consistent temporal scales, reducing the value of xLSTM’s adaptive memory mechanisms. The evidence suggests mainly computational rather than neurophysiological constraints drive the window-dependent patterns—each architecture shows different optima, suggesting patterns reflect architectural limitations rather than underlying neurophysiology.

6.2 Statistical Significance and Clinical Limitations

6.2.1 Statistical Power and Sample Size

The lack of statistical significance in most performance comparisons ($p > 0.05$) is an important limitation. Only two comparisons achieved significance: AUCPR Macro at 5 s ($p=0.005$, t-test) and overall accuracy at 9 s ($p=0.029$, t-test). Balanced accuracy—the primary metric—showed no significant differences. This suggests that training dynamics and optimization problems may be more important than architectural improvements for biosignal processing.

With only 5 runs per configuration, the study lacks enough statistical power to detect small but potentially meaningful differences. The large standard deviations (3-5% for balanced accuracy) show that training dynamics, initialization, and dataset characteristics have a bigger impact on performance than architectural choices. The 4.4% relative improvement (2.18 percentage points absolute) achieved by sLSTM, while consistent, may not be clinically meaningful given overlapping confidence intervals.

6.2.2 Class Imbalance and Deployment Barriers

The confusion matrix analysis reveals severe limitations that question clinical viability:

Severe Class Imbalance Impact: Class 0, representing only 1.93% of training data, never exceeds 15% recall even in the best case (sLSTM at 9 s: $14.2\% \pm 9.4\%$). This means algorithms would miss approximately 85% of minority-class events. In clinical applications, missing any seizure type could have severe patient safety implications. This makes them unsuitable for clinical decision support without additional strategies such as cost-sensitive learning [4], focal loss [16], or targeted data augmentation.

Majority Class Bias: The 87.87% accuracy on Class 5 (66.77% of data) versus 0% accuracy on Class 0 (1.93% of data) shows classic imbalanced learning problems. The wide range of class-specific accuracies (0-87.87%) suggests the models have not learned generalizable seizure detection patterns but rather dataset-specific biases.

Cost-Benefit Analysis: At 9 s windows, training is 60% slower than 5 s windows, with high variance (9.4% standard deviation) and marginal absolute performance (14.2% recall). The substantial computational investment gives minimal improvement on the most critical minority class, indicating future work should prioritize class-imbalance mitigation strategies rather than architectural complexity.

Variability Concerns: The high inter-run variance (coefficient of variation 6-10%) raises serious concerns about deployment reliability. Clinical monitoring systems require consistent, predictable performance, but the substantial variance suggests model behavior may vary significantly depending on initialization and training dynamics.

6.3 Study Limitations

This study has several fundamental limitations that constrain interpretation and applicability:

6.3.1 Methodological Limitations

Sample Size: Five runs per configuration provide insufficient statistical power for robust conclusions, particularly given the observed variance levels. Power analysis suggests 20-30 runs would be needed to detect 2-3 percentage point differences with 80% power.

Single Dataset: Evaluation on only the TUEV dataset limits generalizability. The specific characteristics—particularly severe class imbalance (66.77% majority class, 1.93% rarest class)—heavily influence results. Cross-dataset validation is essential before drawing broad conclusions.

Hyperparameter Optimization: Using hyperparameters optimized for the baseline architecture across all variants may underestimate xLSTM potential. However, exhaustive hyperparameter search would require much more computational resources than available.

6.3.2 Architectural Limitations

Limited Transfer from NLP: The modest improvements compared to dramatic xLSTM success in natural language processing indicate biosignal sequence modeling may require domain-specific architectural innovations rather than adaptations from other fields.

Temporal Window Constraints: The evaluation was restricted to 5, 7, and 9 seconds, which may not capture the full range of temporal contexts relevant for different neurological events. Finer-grained window exploration could better characterize the U-shaped and inverted-U patterns observed.

6.3.3 Dataset and Evaluation Limitations

Class Distribution: The severe class imbalance observed suggests performance would likely vary significantly across datasets with different class distributions.

Computational Constraints: Limited computational resources prevented exploration of larger model configurations, extended training regimes, and comprehensive hyperparameter sweeps.

6.4 Summary

This investigation reveals both the potential and significant limitations of integrating xLSTM architectures within the BIOT framework for EEG event classification. While xLSTM variants achieve window-specific improvements—sLSTM at 5s and 9s windows, mLSTM at 7s windows—the lack of statistical significance for balanced accuracy and critical class-specific failures highlight fundamental challenges that extend beyond architectural innovation. The window-dependent performance patterns, where sLSTM underperforms at 7s and mLSTM degrades at 9s, show that successful approaches from natural language processing do not directly transfer to biosignal analysis. Only the combined architecture provides consistent stability across windows, though it never achieves best-in-class performance.

Most importantly, this work provides a realistic assessment of current limitations and emphasizes the need for honest reporting of window-specific rather than universal improvements, focus on fundamental challenges like class imbalance, larger-scale evaluation

with robust statistical analysis, and recognition that incremental architectural improvements alone may be insufficient for clinical impact. Future research should prioritize addressing these fundamental limitations rather than pursuing incremental architectural modifications.

Chapter 7

Conclusion and Outlook

This thesis investigated the integration of xLSTM architectures within the BIOT framework for automated EEG event classification. The evaluation included sLSTM, mLSTM, and combined variants across three temporal window configurations (5, 7, and 9 seconds), totaling 60 experimental runs (4 architectures $\times$ 3 windows $\times$ 5 random seeds) on the TUEV dataset.

7.1 Summary of Findings

The experimental evaluation reveals that architectural sophistication does not directly translate to superior biosignal performance. As detailed in Section 6.1, each architecture shows distinct window-length preferences rather than consistent superiority: sLSTM demonstrates a U-shaped response with best performance at 5 s and 9 s windows but dips at 7 s, while mLSTM peaks at 7 s but degrades at 9 s. The Linear Transformer baseline shows continuous degradation with increasing window length.

Despite these consistent trends, most differences fail to reach statistical significance (Section 6.2.1). Only two comparisons achieved significance: sLSTM improved AUCPR Macro at 5 s windows ($p=0.005$) and overall accuracy at 9 s windows ($p=0.029$). The primary metric, balanced accuracy, showed no significant differences across all configurations. High inter-run variance (coefficient of variation 6-10%) combined with limited sample size ($n=5$) prevents definitive conclusions about architectural superiority.

The most critical limitation emerges from confusion matrix analysis (Section 6.2.2): severe class imbalance renders all architectures clinically questionable. Class 0 achieves maximum recall of 14.2%, missing approximately 85% of minority-class instances, while Class 5 achieves 81-83% recall, demonstrating strong majority-class bias. These findings indicate that training stability, sample size constraints, and dataset characteristics play a larger role than memory mechanism design.

7.2 Key Contributions

This work makes several important contributions to the field:

- **Comprehensive window-dependent characterization:** Systematic evaluation reveals architecture-specific temporal sensitivities—sLSTM’s U-shaped pattern, mLSTM’s inverted-U response, and the combined architecture’s stability—providing empirical guidance for temporal context selection.

- **Demonstration that training variance dominates architectural effects:** The high coefficient of variation (6-10%) and lack of statistical significance reveal that optimization stability and random initialization have greater impact than architectural sophistication.
- **Transparent, reproducible evaluation pipeline:** This work provides an honest assessment of both achievements and limitations within the BIOT framework [20], including complete reporting of negative results and methodological constraints.

7.3 Limitations

As discussed in Section 6.3, this study faces three major constraints:

Sample Size and Statistical Power: Five runs per configuration provide insufficient power to detect small but potentially meaningful differences. Many observed trends might achieve significance with larger samples (20-30 runs).

Single-Dataset Evaluation: Evaluation on only the TUEV dataset limits generalizability. The specific characteristics—particularly severe class imbalance—heavily influence results. Whether observed architectural patterns generalize to other EEG tasks remains unknown.

Shared Hyperparameters: Using hyperparameters optimized for the baseline across all variants may substantially underestimate xLSTM potential. Whether the modest gains reflect fundamental limitations or simply suboptimal hyperparameter choices remains an open question.

7.4 Future Research Directions

7.4.1 Priorities

Architecture-specific hyperparameter optimization: Systematic tuning for each xLSTM variant is the most critical next step. A comprehensive search should explore dropout rates (0.1-0.4), number of blocks (4-16), combined architecture ratios (beyond the 7:1 configuration), embedding dimensions (128-512), and learning rate schedules. This would clarify whether modest improvements reflect true architectural limitations or suboptimal configurations. Given the window-dependent patterns observed, optimization may need to be performed separately for each window length.

Increased sample sizes: Expanding from 5 to 20-30 runs per configuration would dramatically improve statistical power and enable detection of smaller but clinically meaningful differences. With observed variance levels, many non-significant trends would likely achieve significance with larger samples. This is particularly critical for medical applications where regulatory approval requires robust statistical evidence. Ensemble approaches combining multiple runs could also reduce deployment variance and improve reliability.

Cross-dataset validation: Evaluation should extend to diverse EEG datasets spanning different tasks (sleep staging, seizure detection, motor imagery), recording conditions (clinical vs. ambulatory), and populations (pediatric, adult, elderly). This would establish whether window-dependent patterns represent fundamental architectural characteristics or dataset-specific artifacts. Datasets with different class distributions would help separate architectural effects from imbalance-related issues.

7.4.2 Methodological Advances

Class imbalance mitigation: The complete failure on minority classes demands targeted solutions beyond architectural modifications. Future work should systematically evaluate focal loss [16], cost-sensitive learning [4], oversampling strategies (SMOTE [3], ADASYN [11]), class-balanced batching, and two-stage training approaches. These strategies address fundamental data challenges that architectural improvements alone cannot overcome.

Variance reduction techniques: The high inter-run variance limits deployment reliability. Advanced initialization strategies [7, 12], Stochastic Weight Averaging [14], gradient clipping [18], and multiple checkpoint ensembles could stabilize training and improve generalization. Different memory mechanisms may benefit from different optimization strategies—for example, mLSTM’s matrix memory might need different learning rate schedules than sLSTM’s scalar gates.

Finer-grained window exploration: Testing intermediate windows (6, 8 seconds) and a broader range (3-12 seconds) would more precisely characterize the U-shaped and inverted-U patterns. This could reveal whether certain architectures benefit from very short (2-3 seconds) or very long (12-15 seconds) contexts not evaluated here, and whether the patterns represent continuous curves or artifacts of the specific test points.

7.4.3 Long-Term Directions

Domain-specific memory mechanisms: Rather than directly applying xLSTM from language modeling, future work could develop memory mechanisms specifically designed for biosignal characteristics. EEG exhibits different temporal dynamics than text—periodic oscillations, transient events, and spatially correlated channels. Custom gating mechanisms or memory structures could be more effective. For example, frequency-aware memory cells that explicitly model oscillatory patterns or multi-scale memory with separate mechanisms for fast transients versus slow trends.

Hybrid and adaptive architectures: The complementary window-length sensitivities suggest potential for dynamic architectures that adaptively select or blend memory mechanisms based on sequence characteristics. Gating mechanisms that route information through sLSTM versus mLSTM pathways depending on local pattern characteristics could combine their strengths more effectively than fixed combinations.

Clinical deployment considerations: Real-time clinical monitoring requires low-latency inference, model compression [9] (pruning, quantization). Multi-modal biosignal integration (EEG, ECG, Electromyography (EMG)) could provide richer context. Interpretability features—attention visualization, memory inspection, prototype-based approaches—would enable clinical validation and identify failure modes more effectively than blind parameter tuning.

Ultimately, bridging the gap between model sophistication and clinical relevance requires focusing on stability, transparency, and bias mitigation rather than marginal accuracy improvements. The path forward demands comprehensive methodology combining larger-scale evaluation, architecture-specific optimization, cross-dataset validation, and targeted solutions for fundamental data challenges.

7.5 Final Reflection

This thesis demonstrates that reliable EEG-based event classification depends less on architectural novelty than on rigorous methodology and clinical realism. While xLSTM mechanisms provide new tools for temporal modeling, their value in biosignal processing emerges only when combined with domain-specific design, robust evaluation, and careful attention to medical relevance.

As discussed in Section 6.4, the window-dependent performance patterns suggest no single architecture dominates across all temporal scales. The lack of statistical significance despite consistent trends reveals that training dynamics have greater impact than architectural sophistication. Most critically, severe class imbalance demonstrates that architectural improvements alone cannot overcome fundamental data challenges.

By providing transparent reporting of both achievements and limitations—including negative results, sample size constraints, and potential hyperparameter biases—this work establishes a realistic foundation for future research. The modest improvements observed should not discourage further investigation, but rather guide it toward more comprehensive methodological approaches that can definitively establish the value (or limitations) of advanced sequence modeling in biosignal processing.

The ultimate measure of success in medical machine learning is not marginal performance gains on benchmark metrics, but reliable, interpretable, and clinically actionable predictions. This thesis provides the empirical foundation and methodological framework for pursuing that goal.

Bibliography

- [1] Hubert Banville, Sean U.N. Wood, Chris Aimone, Denis-Alexander Engemann, and Alexandre Gramfort. 2022. Robust learning from corrupted EEG with dynamic spatial filtering. *NeuroImage*, 251, 118994. ISSN: 1053-8119. DOI: 10.1016/j.neuroimage.2022.118994. <https://www.sciencedirect.com/science/article/pii/S1053811922001239>.
- [2] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. 2024. xLSTM: Extended Long Short-Term Memory. *arXiv preprint arXiv:2405.04517*.
- [3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357. DOI: 10.1613/jair.953.
- [4] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. 2019. Class-Balanced Loss Based on Effective Number of Samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9268–9277. DOI: 10.1109/CVPR.2019.00949.
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. (2021). arXiv: 2010.11929 [cs.CV]. <https://arxiv.org/abs/2010.11929>.
- [6] Sean Ferrell, Vineetha Mathew, Matthew Refford, Vincent Tchiong, Tameem Ahsan, Iyad Obeid, and Joseph Picone. 2022. The Temple University Hospital EEG Corpus: Electrode Location and Channel Labels. *Institute for Signal and Information Processing Report*, 1, 1. <https://par.nsf.gov/biblio/10199699>.
- [7] Xavier Glorot and Yoshua Bengio. 2010. Understanding the Difficulty of Training Deep Feedforward Neural Networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256.
- [8] Yuan Gong, Yu-An Chung, and James Glass. 2021. AST: Audio Spectrogram Transformer. (2021). arXiv: 2104.01778 [cs.SD]. <https://arxiv.org/abs/2104.01778>.
- [9] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *International Conference on Learning Representations (ICLR)*. arXiv:1510.00149.
- [10] A. Harati, M. Golmohammadi, S. Lopez, I. Obeid, and J. Picone. 2015. Improved EEG Event Classification Using Differential Energy. In *Proceedings of the IEEE Signal Processing in Medicine and Biology Symposium*. Philadelphia, Pennsylvania, USA, pp. 1–4. https://www.isip.piconepress.com/publications/conference_proceedings/2015/ieee_spmmb/denergy/.

- [11] Haibo He, Yang Bai, Eduardo A. Garcia, and Shutao Li. 2008. ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. In *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*, pp. 1322–1328. DOI: 10.1109/IJCNN.2008.4633969.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation*, 9, 8, 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [14] Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. 2018. Averaging Weights Leads to Wider Optima and Better Generalization. In *Proceedings of the 34th Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 876–885.
- [15] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, volume 119)*. PMLR, pp. 5156–5165.
- [16] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988. DOI: 10.1109/ICCV.2017.324.
- [17] I. Obeid and J. Picone. 2016. The Temple University Hospital EEG Data Corpus. *Frontiers in Neuroscience*, 10, 196. DOI: 10.3389/fnins.2016.00196.
- [18] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the Difficulty of Training Recurrent Neural Networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pp. 1310–1318.
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*. Volume 30.
- [20] Chaoqi Yang, M Brandon Westover, and Jimeng Sun. 2023. BIOT: Biosignal Transformer for Cross-data Learning in the Wild. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=c2LZyTyddi>.

Appendix A

Appendix

A.1 Research Questions and Answers

This section addresses key questions about the research design, execution, and interpretation of results.

A.1.1 What is the central question?

Can xLSTM architectures (sLSTM, mLSTM, and their combination) effectively replace Linear Transformers in the BIOT framework to improve EEG event classification performance, particularly when processing sequences of varying temporal window lengths?

The question specifically investigates whether the enhanced memory mechanisms of xLSTM—scalar memory with exponential gating in sLSTM and matrix memory with key-value associations in mLSTM—provide advantages over linear attention for modeling temporal dependencies in biosignal data.

A.1.2 Why is this question important?

This question addresses a critical gap in biosignal processing research. While deep learning models have achieved impressive results in natural language processing through architectures like xLSTM, their applicability to biosignal analysis remains unclear. EEG signals present unique challenges: severe class imbalance, multi-channel spatial-temporal patterns, and variable recording configurations that differ fundamentally from language sequences.

The question is important for several reasons:

Clinical Relevance: Automated EEG event classification assists neurologists in analyzing long-term recordings for seizure detection, epilepsy monitoring, and neurological disorder diagnosis. Improving classification accuracy, especially for rare but clinically critical events, could enhance patient care.

Domain Transfer Understanding: xLSTM architectures demonstrated substantial improvements in language modeling, but whether these benefits transfer to biosignal domains with fundamentally different sequence characteristics (frequency-domain tokens vs. discrete words, local patterns vs. long-range dependencies) is unknown.

Temporal Context Optimization: The optimal temporal window size for capturing relevant neurological context around events is not well established. Understanding how different architectures respond to varying context lengths provides insights into both architectural capabilities and neurophysiological patterns.

A.1.3 What evidence/data (variables) are needed to answer this question?

The evaluation requires four categories of evidence:

Performance metrics must capture classification quality under severe class imbalance (66.77% majority, 1.93% rarest class):

- Balanced accuracy (primary metric equally weighting all six event classes)
- Cohen’s kappa (agreement beyond chance)
- Weighted F1 score
- AUCPR macro (ranking quality for rare events)

Architectural variants serve as independent variables:

- Linear Transformer baseline
- sLSTM
- mLSTM
- Combined mLSTM+sLSTM (7:1 ratio)

Temporal window configurations test architecture sensitivity to sequence length: 5 s, 7 s, and 9 s windows.

Training dynamics data assess stability and reliability:

- Standard deviation across five random seeds (10, 42, 123, 456, 999)
- Computational runtime
- Statistical significance measures (paired t-test and Wilcoxon p-values)
- Class-specific performance from normalized confusion matrices (per-class recall, systematic misclassification patterns)

The TUEV dataset provides the substrate: 359 training files and 159 evaluation files spanning six event classes (SPSW, GPED, PLED, EYEM, ARTF, BCKG) with 16-channel bipolar TCP montage at 200 Hz.

A.1.4 What methods are used to get this evidence/data?

The experimental design systematically compares 4 architectures $\times$ 3 window lengths $\times$ 5 random seeds totaling 60 runs. All variants share fixed hyperparameters (dropout 0.2, 8 blocks, embedding dimension 256, batch size 128) to isolate architectural effects. The training set (359 files) is partitioned 90% train, 10% validation at the subject level, with a separate evaluation set (159 files) for testing. Checkpoint selection uses balanced accuracy rather than validation loss to address class imbalance.

Implementation uses PyTorch 2.x with PyTorch Lightning and the official xlstm package on a single NVIDIA RTX 3090 GPU (24GB VRAM). Training runs 100 epochs with AdamW optimizer (learning rate 1×10^{-4}) and early stopping. The preprocessing pipeline:

- Converts unipolar recordings to 16-channel bipolar TCP montage
- Resamples from 250 Hz to 200 Hz using anti-aliasing filters
- Applies 95th percentile normalization for outlier robustness

- Tokenizes via STFT (FFT size 200, hop length 100, Hann window)

Evaluation combines PyHealth metrics computation, paired statistical tests (t-test and Wilcoxon signed-rank), normalized confusion matrices averaged over seeds and runtime measurement. Reproducibility measures include fixed random seeds and in depth documentation of experiments.

A.1.5 What analyses must be applied to the data to answer the central question?

Six complementary analyses address the research question:

Performance comparison computes mean and standard deviation across seeds for each architecture-window combination, identifies best performers per window, and calculates coefficient of variation for stability assessment.

Statistical significance testing applies paired t-tests and Wilcoxon tests ($\alpha = 0.05$) comparing best xLSTM variants against baseline at each window length, reporting both p-values and effect sizes while cautiously interpreting results given limited sample size ($n = 5$).

Window-length dependency analysis plots metric trajectories across windows to identify U-shaped, inverted-U, or monotonic patterns that reveal architecture-specific sequence-length sensitivities and scaling behaviors.

Class-specific performance analysis generates normalized confusion matrices averaged over seeds, extracts per-class recall from diagonal entries, identifies systematic misclassification patterns from off-diagonal entries, and correlates performance with class frequency (1.93% to 66.77%) to assess clinical viability.

Computational efficiency analysis measures training runtime with variance across seeds, calculates speedup factors relative to xLSTM variants, assesses scaling with window length (linear, sublinear, superlinear), and evaluates performance-efficiency trade-offs.

Stability analysis compares standard deviations across architectures for optimization stability, identifies variance patterns where $\sigma > \text{mean}$ indicates unstable predictions, and evaluates deployment reliability from cross-seed variability.

A.1.6 What evidence/data (values for the variables) were obtained?

Balanced accuracy results reveal window-dependent performance patterns:

- **5 s windows:** sLSTM achieves $52.15\% \pm 1.37\%$, nearly matching Linear Transformer's $52.06\% \pm 4.06\%$
- **7 s windows:** mLSTM leads ($50.66\% \pm 2.87\%$) while sLSTM drops significantly ($48.45\% \pm 5.50\%$)
- **9 s windows:** sLSTM rebounds ($52.46\% \pm 3.81\%$) as Linear Transformer degrades ($48.45\% \pm 3.73\%$)
- **Combined:** Stable across windows (50.03–50.46%) but never best-in-class

Statistical testing shows balanced accuracy never reached significance at any window length. Only secondary metrics achieved significance: AUCPR Macro at 5 s ($p = 0.005$) and overall accuracy at 9 s ($p = 0.029$). Training runtime ranges from 1h49m–3h15m

for Linear Transformer ($2.3\text{--}3.4\times$ faster than xLSTM), 4h10m–6h40m for sLSTM, to 5h30m–10h57m for mLSTM with superlinear scaling.

Class-specific recall reveals severe imbalance effects: Class 0 (1.93% frequency) achieves only 0–14.2% recall (clinical failure), while Class 5 (66.77% frequency) dominates with 79.4–87.87% recall. Intermediate classes achieve 41–66% recall. Architecture-specific behaviors show Linear Transformer with monotonic degradation ($52.06\% \rightarrow 50.08\% \rightarrow 48.45\%$), sLSTM with U-shaped pattern, and mLSTM with inverted-U pattern. Cross-seed variability is high (CV 6–10%), with Class 0 variance exceeding mean ($\sigma = 16.4\%$, mean = 8.2%), indicating unstable predictions.

A.1.7 What were the results of the analyses?

Performance comparison shows that xLSTM variants did not consistently outperform the baseline:

- sLSTM excelled at 5 s and 9 s but failed at 7 s
- mLSTM peaked at 7 s but collapsed at 9 s (46.40%)
- Linear Transformer remained competitive (52.06% at 5 s) with $2.3\times$ faster training
- Combined architecture provided stable but mediocre performance

Small effect sizes ($< 5\%$) and high variability (CV 6–10%) suggest training dynamics matter more than architecture.

Statistical testing showed balanced accuracy never reached significance (all $p > 0.11$). Only secondary metrics were significant: AUCPR at 5 s ($p = 0.005$) and overall accuracy at 9 s ($p = 0.029$). With only $n = 5$ runs, the statistical power is too low to detect small differences.

Window-length patterns revealed architecture-specific behaviors. Linear Transformer degraded monotonically across windows. sLSTM showed U-shaped performance with Class 2 recall dropping from 52.5% to 42.9% at 7 s ($p = 0.043$). mLSTM showed inverted-U pattern with collapse at 9 s. Combined architecture maintained stability (50.03–50.46%) but never reached best performance.

Class-specific analysis confirmed severe imbalance effects. Class 0 (1.93% frequency) achieved only 0–14.2% recall (clinically unusable). Class 5 (66.77% frequency) dominated with 79.4–87.87% recall. xLSTM provided minimal improvements beyond slightly better majority-class stability.

Computational costs were substantial. xLSTM variants require $2.3\text{--}3.4\times$ longer training, with mLSTM showing superlinear scaling. Combined with modest gains ($< 5\%$), this indicates poor efficiency trade-offs.

Stability varied across architectures. sLSTM reduced variance at 5 s (1.37% vs. 4.06%) but increased it at 7 s (5.50%). Combined architecture achieved good stability (CV 1.1–2.8%). However, Class 0 predictions remained unstable, and high overall variability (CV 6–10%) raises deployment concerns.

A.1.8 How did the analyses answer the central question?

The analyses provide a nuanced answer to the central question: **xLSTM architectures can replace Linear Transformers in the BIOT framework, but they do**

not consistently improve EEG event classification performance and introduce significant computational costs.

Key Findings:

1. Window-Specific, Not Universal, Improvements. xLSTM benefits depend strongly on window length. sLSTM works best at 5 s and 9 s, while mLSTM excels at 7 s. This shows that xLSTM’s memory mechanisms only help under specific conditions—not universally like in language processing.

2. Limited Domain Transfer from NLP. The modest improvements ($< 5\%$) contrast with xLSTM’s strong gains in language modeling. Language sequences have long-range dependencies over hundreds of tokens, while biosignals show mostly local patterns in 9–17 tokens per channel. The enhanced memory designed for language may be overengineered for neurological events.

3. Training Dynamics Dominate Architecture. High variability (CV 6–10%) and lack of statistical significance show that training dynamics and initialization matter more than architecture. Improving training stability and addressing class imbalance may be more effective than pursuing complex architectures.

4. Class Imbalance Remains Unaddressed. Neither xLSTM nor baseline achieve useful performance on minority classes. Class 0 never exceeds 15% recall, making clinical deployment impossible without addressing imbalanced learning (e.g., cost-sensitive loss, data augmentation, focal loss).

5. Computational Cost Outweighs Benefits. xLSTM variants need $2.3\text{--}3.4\times$ more training time. Given modest, inconsistent gains and no statistical significance, this computational cost provides poor practical value.

6. Combined Architecture: Stable but Not Best. The mLSTM+sLSTM architecture (7:1 ratio) achieves good stability (50.03–50.46%) but never reaches best performance. The fixed ratio dilutes individual strengths rather than combining them effectively.

Direct Answer: xLSTM architectures *can* replace Linear Transformers but *should not* in most cases. The improvements are too inconsistent, computational costs too high, and class imbalance remains unsolved. Deployment would need careful window selection, $2\text{--}3\times$ longer training, and additional strategies for minority classes—making the baseline more practical.

A.1.9 What does this answer tell us about the broader field?

The findings have important implications for biosignal processing research, domain transfer in deep learning, and clinical machine learning deployment:

1. Domain Transfer Is Not Automatic. Successful Natural Language Processing (NLP) architectures do not automatically work for biosignals. Biosignal data differs fundamentally: continuous tokens, frequency-domain content, and local patterns rather

than long-range dependencies. Researchers should validate NLP models empirically before assuming they transfer to medical signals.

2. Sequence Length and Architecture Interact. The U-shaped and inverted-U patterns show that architecture effectiveness depends on sequence length. One-size-fits-all architectures may be suboptimal. Adaptive architectures that adjust memory based on sequence length could provide more consistent benefits.

3. Class Imbalance Dominates Architecture. Architectural sophistication cannot fix data imbalance. The 0–15% recall on minority classes shows that addressing imbalance (sampling, loss design, augmentation) should be prioritized over architectural improvements. For clinical Machine Learning (ML), data-centric approaches may work better than model-centric ones.

4. Training Stability Matters More Than Expected. High variability (CV 6–10%) shows that training dynamics impact results as much as architecture. Research should focus on better initialization, stable optimization, ensemble methods, and larger studies (> 20 runs).

5. Computational Efficiency Is Critical. The $2.3\text{--}3.4\times$ training time increase with modest gains shows that efficiency should be evaluated alongside accuracy. This matters especially for clinical deployment where resources and energy costs are limited.

6. Statistical Rigor Is Often Lacking. No statistical significance despite numerical differences shows the need for larger sample sizes (20–30 runs), proper corrections, confidence intervals, and honest reporting. Many papers use only 3–5 runs without statistical testing, potentially overstating small, unstable improvements.

7. Clinical Viability Needs More Than Accuracy. Severe minority-class failures (85–100% missed for Class 0) despite decent aggregate metrics (52% balanced accuracy) show that standard metrics can hide clinical problems. Evaluation should include:

- Class-specific performance
- Clinical costs (false negative vs. false positive)
- Deployment reliability (variance, worst-case)
- Computational feasibility (time, latency, resources)

8. Tokenization May Matter More Than Architecture. The short sequences (9–17 tokens) and good baseline performance suggest that tokenization (converting 0.5-second windows to frequency tokens) may be more important than the sequence model. Research might focus better on improving tokenization (learnable bands, adaptive windows) rather than complex sequence models.

9. Negative Results Are Valuable. This work shows the value of reporting all results (not cherry-picking), acknowledging insignificance, and highlighting limitations. The field needs more honest reporting when advanced architectures fail to improve consistently.

Future Research Directions:

The findings suggest several high-priority research directions:

- **Class imbalance mitigation:** Cost-sensitive learning, focal loss, data augmentation specifically for minority classes
- **Training stability improvement:** Better initialization, regularization, ensemble methods

- **Domain-specific architectures:** Biosignal-specific designs incorporating spatial-temporal characteristics
- **Tokenization innovation:** Learnable, adaptive frequency-domain representations
- **Cross-dataset validation:** Evaluating generalization to multiple hospitals, recording systems, patient populations
- **Larger-scale evaluation:** 20–30 runs per configuration for robust statistical conclusions
- **Adaptive window selection:** Task-specific or event-specific temporal context optimization

Appendix B

Appendix

B.1 Additional Metric Figures

This section provides additional per-metric figures for the architecture comparison across the three temporal windows (5s, 7s, 9s). In the main text (Chapter 5), we focus the figure discussion on AUCPR Macro to keep the presentation compact and to emphasize the metric that is most informative for rare-event ranking.

All plots below use boxplots over the five random seeds. This makes the run-to-run variability easier to compare than overlapping mean $\pm$ std curves.

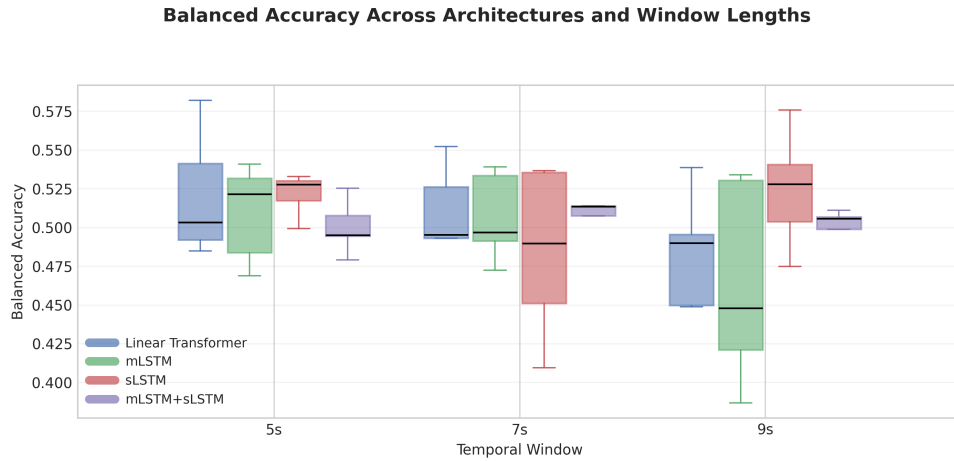


Figure B.1: Balanced accuracy across architectures and temporal window configurations (5s, 7s, 9s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture.

B.2 Complete Performance Metrics Per Run

This section provides detailed performance metrics for individual experimental runs across all architectures and temporal window configurations. The data supplements the aggregated results presented in Chapter 5 and enables verification of the statistical analyses presented in the main text.

B.2.1 Individual Run Performance Metrics by Architecture

Tables B.1 through B.12 present complete performance metrics for each of the five random seeds (10, 42, 123, 456, 999) across all architectural variants and temporal window

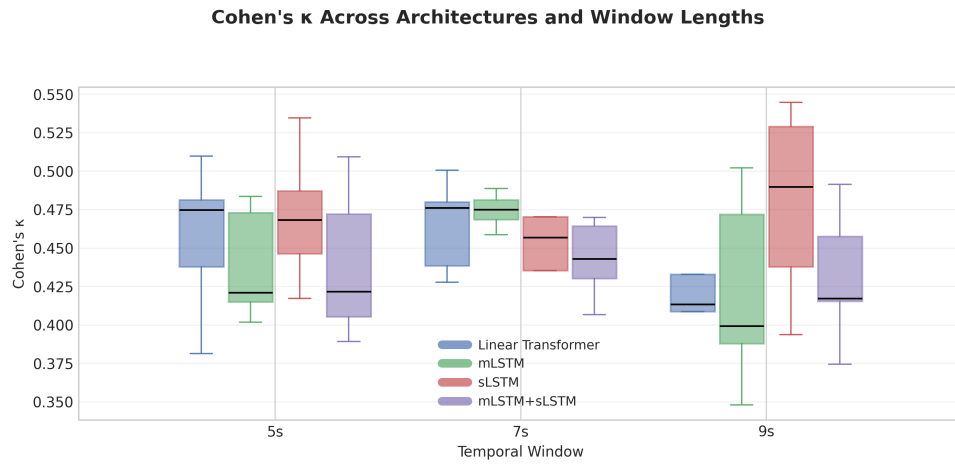


Figure B.2: Cohen's κ across architectures and temporal window configurations (5s, 7s, 9s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture.

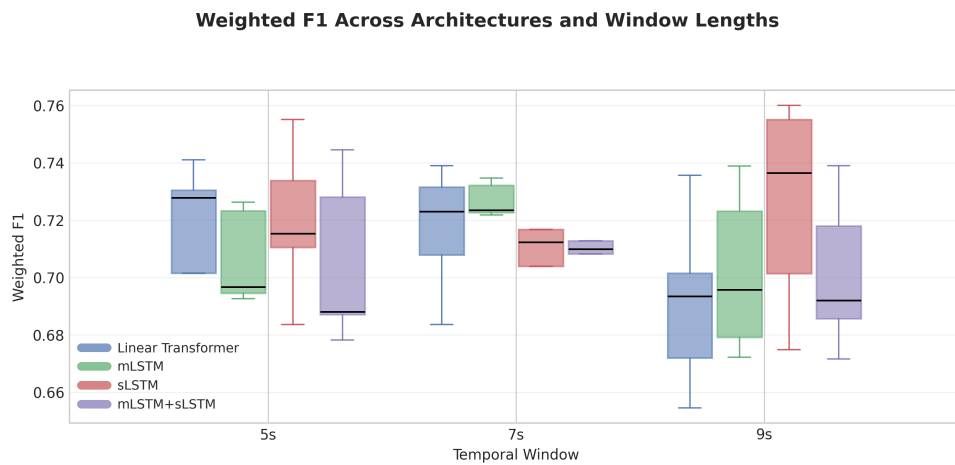


Figure B.3: Weighted F1 across architectures and temporal window configurations (5s, 7s, 9s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture.

configurations. All metrics are computed using balanced-accuracy checkpoint selection as described in Section 5.

Table B.1: Individual run metrics for Linear Transformer at 5 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4920	0.6769	0.4378	0.7015	0.8642	0.8777	0.4537
42	0.5819	0.7098	0.4748	0.7305	0.8714	0.8675	0.4780
123	0.5413	0.7180	0.5097	0.7411	0.8664	0.8938	0.4848
456	0.5032	0.7074	0.4812	0.7278	0.8613	0.8899	0.4629
999	0.4848	0.6308	0.3814	0.6565	0.8697	0.8664	0.4583
Mean	0.5206	0.6886	0.4570	0.7115	0.8666	0.8791	0.4675
Std	0.0406	0.0359	0.0494	0.0340	0.0041	0.0126	0.0133

Table B.2: Individual run metrics for Linear Transformer at 7 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4695	0.6686	0.4546	0.7074	0.8710	0.8960	0.5069
42	0.4841	0.6693	0.4326	0.6993	0.8412	0.8794	0.4302
123	0.5630	0.7120	0.4913	0.7357	0.8672	0.8926	0.4833
456	0.4978	0.6873	0.4709	0.7217	0.8502	0.8869	0.4386
999	0.4898	0.7057	0.4833	0.7332	0.8629	0.8925	0.4674
Mean	0.5008	0.6886	0.4665	0.7195	0.8585	0.8895	0.4653
Std	0.0429	0.0285	0.0296	0.0224	0.0184	0.0084	0.0388

Table B.3: Individual run metrics for Linear Transformer at 9 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4502	0.6686	0.4305	0.7058	0.8601	0.8788	0.4849
42	0.5283	0.7064	0.4844	0.7241	0.8501	0.8624	0.4629
123	0.5008	0.6648	0.4072	0.6850	0.8475	0.8664	0.4317
456	0.5117	0.6738	0.4381	0.6953	0.8440	0.8601	0.4427
999	0.4513	0.6209	0.3521	0.6528	0.8405	0.8551	0.4407
Mean	0.4885	0.6669	0.4225	0.6926	0.8484	0.8646	0.4526
Std	0.0373	0.0361	0.0479	0.0307	0.0101	0.0117	0.0255

Table B.4: Individual run metrics for sLSTM at 5 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.5277	0.6915	0.4463	0.7106	0.8646	0.8706	0.5162
42	0.4993	0.6873	0.4173	0.6836	0.8676	0.8605	0.5031
123	0.5173	0.7501	0.5346	0.7551	0.8834	0.8993	0.5322
456	0.5328	0.7157	0.4872	0.7338	0.8756	0.8881	0.4915
999	0.5302	0.7095	0.4682	0.7153	0.8630	0.8687	0.5205
Mean	0.5215	0.7108	0.4707	0.7197	0.8708	0.8774	0.5127
Std	0.0137	0.0250	0.0442	0.0267	0.0085	0.0158	0.0158

Table B.5: Individual run metrics for sLSTM at 7 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4094	0.6569	0.3585	0.6694	0.8469	0.8389	0.4604
42	0.5355	0.6906	0.4703	0.7168	0.8906	0.8946	0.5305
123	0.5366	0.8032	0.6090	0.7965	0.8781	0.9163	0.5229
456	0.4511	0.6894	0.4353	0.7039	0.8689	0.8803	0.4362
999	0.4898	0.6967	0.4569	0.7123	0.8949	0.9040	0.5142
Mean	0.4845	0.7074	0.4660	0.7198	0.8759	0.8868	0.4928
Std	0.0550	0.0558	0.0909	0.0467	0.0192	0.0298	0.0419

Table B.6: Individual run metrics for sLSTM at 9 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4748	0.6505	0.3936	0.6749	0.8626	0.8657	0.4426
42	0.5279	0.7423	0.4898	0.7365	0.8819	0.9000	0.5384
123	0.5038	0.7472	0.5446	0.7600	0.8496	0.9006	0.4544
456	0.5757	0.7356	0.5289	0.7551	0.8835	0.9072	0.4852
999	0.5407	0.6937	0.4378	0.7014	0.8768	0.8702	0.5068
Mean	0.5246	0.7139	0.4789	0.7256	0.8709	0.8887	0.4855
Std	0.0381	0.0413	0.0630	0.0365	0.0145	0.0193	0.0389

Table B.7: Individual run metrics for mLSTM at 5 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4689	0.7143	0.4018	0.6967	0.8206	0.8226	0.4535
42	0.5318	0.6961	0.4149	0.6926	0.8734	0.8520	0.5297
123	0.4836	0.6929	0.4209	0.6946	0.8194	0.8608	0.4654
456	0.5215	0.7047	0.4730	0.7233	0.8779	0.8770	0.4928
999	0.5408	0.7008	0.4835	0.7263	0.8764	0.8826	0.4938
Mean	0.5093	0.7018	0.4388	0.7067	0.8535	0.8590	0.4870
Std	0.0314	0.0083	0.0368	0.0166	0.0307	0.0238	0.0295

Table B.8: Individual run metrics for mLSTM at 7s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4723	0.7016	0.4586	0.7219	0.8220	0.8657	0.4790
42	0.5336	0.7013	0.4751	0.7235	0.8805	0.8779	0.4927
123	0.4969	0.7107	0.4886	0.7322	0.8210	0.8698	0.4581
456	0.5391	0.7194	0.4813	0.7347	0.8623	0.8573	0.4945
999	0.4913	0.7215	0.4684	0.7227	0.8568	0.8677	0.4931
Mean	0.5066	0.7109	0.4744	0.7270	0.8485	0.8677	0.4835
Std	0.0287	0.0095	0.0116	0.0060	0.0262	0.0074	0.0155

Table B.9: Individual run metrics for mLSTM at 9s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4479	0.6845	0.3879	0.6792	0.8272	0.8486	0.4331
42	0.3868	0.6846	0.3480	0.6722	0.7900	0.8347	0.4455
123	0.5339	0.6935	0.4718	0.7232	0.8232	0.8711	0.5009
456	0.5304	0.7186	0.5021	0.7389	0.8790	0.8843	0.4944
999	0.4210	0.7077	0.3992	0.6958	0.8055	0.8335	0.4215
Mean	0.4640	0.6978	0.4218	0.7019	0.8250	0.8544	0.4591
Std	0.0659	0.0150	0.0634	0.0285	0.0337	0.0225	0.0363

Table B.10: Individual run metrics for mLSTM+sLSTM at 5s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4790	0.6815	0.3892	0.6783	0.8518	0.8562	0.5005
42	0.5252	0.7300	0.5092	0.7445	0.8606	0.8789	0.5039
123	0.4944	0.6939	0.4054	0.6871	0.8520	0.8444	0.4850
456	0.5076	0.6598	0.4217	0.6881	0.8624	0.8667	0.4716
999	0.4951	0.7239	0.4720	0.7281	0.8762	0.8708	0.4827
Mean	0.5003	0.6978	0.4395	0.7052	0.8606	0.8634	0.4887
Std	0.0172	0.0293	0.0498	0.0292	0.0100	0.0134	0.0133

Table B.11: Individual run metrics for mLSTM+sLSTM at 7s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.5139	0.6712	0.4066	0.6799	0.8642	0.8535	0.4609
42	0.5074	0.6984	0.4430	0.7100	0.8523	0.8595	0.4793
123	0.5315	0.6989	0.4643	0.7128	0.8596	0.8811	0.5127
456	0.5135	0.7238	0.4698	0.7242	0.8508	0.8771	0.4737
999	0.4568	0.7087	0.4301	0.7083	0.8450	0.8436	0.4677
Mean	0.5046	0.7002	0.4428	0.7070	0.8544	0.8630	0.4789
Std	0.0282	0.0192	0.0258	0.0164	0.0076	0.0159	0.0201

Table B.12: Individual run metrics for mLSTM+sLSTM at 9 s windows.

Seed	Bal. Acc.	Accuracy	Cohen κ	F1 (W)	AUROC (M)	AUROC (W)	AUCPR (M)
10	0.4822	0.6884	0.3744	0.6716	0.8472	0.8310	0.4760
42	0.5057	0.7099	0.4576	0.7180	0.8595	0.8719	0.4966
123	0.5111	0.6670	0.4155	0.6857	0.8430	0.8580	0.4575
456	0.5069	0.7297	0.4913	0.7390	0.8663	0.8739	0.4888
999	0.4989	0.6990	0.4173	0.6921	0.8444	0.8390	0.4419
Mean	0.5010	0.6988	0.4312	0.7013	0.8521	0.8548	0.4722
Std	0.0114	0.0234	0.0447	0.0270	0.0103	0.0193	0.0225

B.2.2 Complete Experimental Runtime Data

Table B.13 presents the complete training runtime for each individual run across all experimental configurations. All runs completed 100 training epochs with identical hardware (NVIDIA RTX 3090) and batch size (128).

Table B.13: Complete training runtimes for individual runs (100 epochs each). Times are in hours:minutes format.

Architecture	Window	Seed 10	Seed 42	Seed 123	Seed 456	Seed 999
Linear Transformer	5s	1h 47m	1h 54m	1h 50m	1h 47m	1h 49m
	7s	2h 39m	2h 37m	2h 39m	2h 38m	2h 38m
	9s	3h 16m	3h 16m	3h 16m	3h 15m	3h 15m
mLSTM	5s	5h 25m	5h 23m	5h 26m	5h 47m	5h 32m
	7s	7h 42m	7h 50m	7h 45m	7h 54m	7h 54m
	9s	10h 28m	11h 12m	11h 11m	11h 12m	11h 00m
sLSTM	5s	4h 21m	4h 03m	3h 58m	4h 12m	4h 19m
	7s	5h 29m	5h 32m	5h 28m	5h 39m	5h 36m
	9s	6h 32m	6h 48m	6h 40m	6h 42m	6h 41m
mLSTM+sLSTM	5s	5h 34m	5h 49m	5h 23m	5h 19m	5h 37m
	7s	7h 33m	7h 31m	7h 33m	8h 00m	7h 51m
	9s	10h 27m	10h 19m	10h 23m	10h 38m	10h 38m

B.3 Complete Confusion Matrix Collection

This section provides normalized confusion matrices for all architectural variants and temporal window configurations that were not included in the main results chapter. These matrices enable comprehensive analysis of class-specific performance patterns. Confusion matrices are normalized row-wise, so diagonal entries represent per-class recall (sensitivity) and off-diagonal entries show misclassification rates. All values are mean $\pm$ standard deviation across five random seeds.

B.3.1 Overview

A confusion matrix displays the classification performance for each true class (rows) against predicted classes (columns). In this multiclass seizure classification task with six classes (0-5):

- **Diagonal values (bold):** Per-class recall—the fraction of true instances correctly classified
- **Off-diagonal values:** Misclassification rates—the fraction of true instances incorrectly assigned to other classes
- **Class distribution:** Class 0 (1.93%), Class 1 (15.89%), Class 2 (6.79%), Class 3 (1.12%), Class 4 (7.49%), Class 5 (66.77%)

The severe class imbalance strongly influences model behavior, with majority-class bias evident across all architectures.

Table B.14: Normalized Confusion Matrix: Linear Transformer at 5-second windows ($\pm 2s/\pm 2s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.082 ± 0.164	0.259 ± 0.135	0.140 ± 0.109	0.070 ± 0.067	0.123 ± 0.150	0.325 ± 0.082
Class 1	0.027 ± 0.027	0.411 ± 0.119	0.201 ± 0.035	0.010 ± 0.009	0.229 ± 0.114	0.122 ± 0.061
Class 2	0.047 ± 0.022	0.144 ± 0.038	0.567 ± 0.064	0.065 ± 0.041	0.063 ± 0.035	0.113 ± 0.037
Class 3	0.005 ± 0.003	0.002 ± 0.005	0.022 ± 0.019	0.664 ± 0.046	0.086 ± 0.045	0.221 ± 0.034
Class 4	0.007 ± 0.006	0.006 ± 0.005	0.029 ± 0.029	0.038 ± 0.020	0.605 ± 0.099	0.316 ± 0.088
Class 5	0.029 ± 0.018	0.011 ± 0.005	0.047 ± 0.011	0.029 ± 0.014	0.089 ± 0.034	0.794 ± 0.030

Table B.15: Normalized Confusion Matrix: Linear Transformer at 7-second windows ($\pm 3s/\pm 3s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.066 ± 0.089	0.383 ± 0.130	0.086 ± 0.039	0.090 ± 0.043	0.141 ± 0.155	0.234 ± 0.036
Class 1	0.028 ± 0.028	0.485 ± 0.117	0.191 ± 0.070	0.008 ± 0.009	0.235 ± 0.114	0.053 ± 0.017
Class 2	0.079 ± 0.071	0.181 ± 0.096	0.466 ± 0.100	0.088 ± 0.049	0.065 ± 0.016	0.119 ± 0.061
Class 3	0.002 ± 0.004	0.009 ± 0.009	0.016 ± 0.013	0.593 ± 0.082	0.104 ± 0.051	0.275 ± 0.061
Class 4	0.002 ± 0.003	0.011 ± 0.009	0.013 ± 0.011	0.071 ± 0.028	0.606 ± 0.073	0.296 ± 0.092
Class 5	0.022 ± 0.009	0.015 ± 0.014	0.034 ± 0.010	0.033 ± 0.017	0.107 ± 0.037	0.788 ± 0.066

Table B.16: Normalized Confusion Matrix: Linear Transformer at 9-second windows ($\pm 4s/\pm 4s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.017 ± 0.028	0.349 ± 0.121	0.240 ± 0.069	0.078 ± 0.061	0.031 ± 0.048	0.286 ± 0.049
Class 1	0.007 ± 0.005	0.404 ± 0.183	0.270 ± 0.076	0.008 ± 0.008	0.175 ± 0.102	0.135 ± 0.076
Class 2	0.041 ± 0.024	0.162 ± 0.075	0.512 ± 0.117	0.099 ± 0.060	0.053 ± 0.040	0.134 ± 0.068
Class 3	0.009 ± 0.012	0.007 ± 0.010	0.019 ± 0.010	0.616 ± 0.153	0.068 ± 0.048	0.281 ± 0.130
Class 4	0.008 ± 0.005	0.008 ± 0.012	0.022 ± 0.012	0.061 ± 0.041	0.584 ± 0.054	0.317 ± 0.052
Class 5	0.031 ± 0.019	0.012 ± 0.009	0.049 ± 0.018	0.036 ± 0.028	0.099 ± 0.020	0.774 ± 0.044

Table B.17: Normalized Confusion Matrix: sLSTM at 5-second windows ($\pm 2s/\pm 2s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.051 ± 0.078	0.400 ± 0.058	0.238 ± 0.022	0.056 ± 0.023	0.062 ± 0.059	0.194 ± 0.085
Class 1	0.027 ± 0.018	0.420 ± 0.143	0.184 ± 0.015	0.007 ± 0.005	0.108 ± 0.063	0.255 ± 0.090
Class 2	0.093 ± 0.060	0.116 ± 0.062	0.525 ± 0.095	0.047 ± 0.044	0.077 ± 0.025	0.142 ± 0.052
Class 3	0.004 ± 0.008	0.000 ± 0.000	0.012 ± 0.019	0.664 ± 0.120	0.097 ± 0.042	0.224 ± 0.086
Class 4	0.001 ± 0.001	0.010 ± 0.007	0.011 ± 0.006	0.030 ± 0.016	0.643 ± 0.057	0.305 ± 0.053
Class 5	0.017 ± 0.018	0.018 ± 0.007	0.030 ± 0.013	0.011 ± 0.006	0.097 ± 0.020	0.826 ± 0.015

Table B.18: Normalized Confusion Matrix: sLSTM at 7-second windows ($\pm 3s/\pm 3s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.010 ± 0.015	0.384 ± 0.066	0.126 ± 0.123	0.164 ± 0.125	0.081 ± 0.076	0.235 ± 0.097
Class 1	0.017 ± 0.014	0.449 ± 0.166	0.149 ± 0.040	0.006 ± 0.007	0.200 ± 0.116	0.179 ± 0.070
Class 2	0.091 ± 0.046	0.174 ± 0.059	0.429 ± 0.109	0.064 ± 0.052	0.086 ± 0.043	0.155 ± 0.085
Class 3	0.005 ± 0.011	0.004 ± 0.010	0.009 ± 0.019	0.590 ± 0.090	0.083 ± 0.036	0.309 ± 0.083
Class 4	0.004 ± 0.005	0.011 ± 0.014	0.004 ± 0.002	0.018 ± 0.011	0.598 ± 0.167	0.366 ± 0.160
Class 5	0.023 ± 0.011	0.021 ± 0.008	0.013 ± 0.005	0.010 ± 0.010	0.102 ± 0.042	0.832 ± 0.056

Table B.19: Normalized Confusion Matrix: sLSTM at 9-second windows ($\pm 4s/\pm 4s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.142 ± 0.094	0.371 ± 0.144	0.223 ± 0.151	0.096 ± 0.061	0.014 ± 0.022	0.153 ± 0.075
Class 1	0.047 ± 0.058	0.444 ± 0.173	0.238 ± 0.118	0.007 ± 0.010	0.078 ± 0.041	0.185 ± 0.153
Class 2	0.047 ± 0.030	0.170 ± 0.015	0.516 ± 0.089	0.059 ± 0.043	0.061 ± 0.041	0.146 ± 0.071
Class 3	0.020 ± 0.038	0.002 ± 0.004	0.006 ± 0.011	0.638 ± 0.170	0.075 ± 0.080	0.258 ± 0.106
Class 4	0.011 ± 0.008	0.015 ± 0.009	0.015 ± 0.011	0.038 ± 0.039	0.576 ± 0.055	0.345 ± 0.048
Class 5	0.022 ± 0.010	0.029 ± 0.006	0.027 ± 0.017	0.013 ± 0.014	0.078 ± 0.036	0.831 ± 0.045

Table B.20: Normalized Confusion Matrix: mLSTM at 5-second windows ($\pm 2s/\pm 2s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.083 ± 0.072	0.443 ± 0.039	0.078 ± 0.062	0.095 ± 0.048	0.059 ± 0.054	0.242 ± 0.036
Class 1	0.006 ± 0.004	0.395 ± 0.113	0.143 ± 0.061	0.006 ± 0.002	0.138 ± 0.080	0.312 ± 0.221
Class 2	0.022 ± 0.016	0.135 ± 0.026	0.517 ± 0.057	0.048 ± 0.025	0.116 ± 0.022	0.162 ± 0.062
Class 3	0.005 ± 0.011	0.010 ± 0.009	0.013 ± 0.022	0.682 ± 0.030	0.147 ± 0.046	0.144 ± 0.055
Class 4	0.004 ± 0.002	0.016 ± 0.008	0.010 ± 0.008	0.013 ± 0.010	0.550 ± 0.072	0.406 ± 0.061
Class 5	0.014 ± 0.004	0.026 ± 0.007	0.026 ± 0.010	0.004 ± 0.002	0.102 ± 0.031	0.829 ± 0.040

Table B.21: Normalized Confusion Matrix: mLSTM at 7-second windows ($\pm 3s/\pm 3s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.083 ± 0.108	0.425 ± 0.115	0.044 ± 0.031	0.139 ± 0.061	0.019 ± 0.022	0.289 ± 0.053
Class 1	0.030 ± 0.028	0.532 ± 0.091	0.141 ± 0.037	0.010 ± 0.006	0.114 ± 0.078	0.171 ± 0.112
Class 2	0.076 ± 0.053	0.137 ± 0.030	0.483 ± 0.087	0.053 ± 0.028	0.100 ± 0.026	0.150 ± 0.043
Class 3	0.002 ± 0.005	0.007 ± 0.009	0.016 ± 0.016	0.579 ± 0.036	0.160 ± 0.021	0.235 ± 0.052
Class 4	0.005 ± 0.005	0.011 ± 0.010	0.012 ± 0.012	0.008 ± 0.009	0.546 ± 0.077	0.417 ± 0.064
Class 5	0.028 ± 0.017	0.024 ± 0.013	0.020 ± 0.007	0.006 ± 0.003	0.106 ± 0.010	0.815 ± 0.020

Table B.22: Normalized Confusion Matrix: mLSTM at 9-second windows ($\pm 4s/\pm 4s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.015 ± 0.027	0.441 ± 0.036	0.037 ± 0.067	0.124 ± 0.079	0.043 ± 0.048	0.340 ± 0.146
Class 1	0.018 ± 0.013	0.417 ± 0.161	0.142 ± 0.045	0.003 ± 0.003	0.151 ± 0.082	0.269 ± 0.185
Class 2	0.085 ± 0.086	0.149 ± 0.058	0.486 ± 0.121	0.040 ± 0.024	0.058 ± 0.025	0.183 ± 0.075
Class 3	0.005 ± 0.008	0.001 ± 0.003	0.013 ± 0.009	0.604 ± 0.100	0.114 ± 0.033	0.263 ± 0.100
Class 4	0.004 ± 0.004	0.010 ± 0.008	0.009 ± 0.007	0.025 ± 0.011	0.425 ± 0.139	0.528 ± 0.149
Class 5	0.025 ± 0.017	0.024 ± 0.006	0.020 ± 0.007	0.009 ± 0.008	0.084 ± 0.035	0.838 ± 0.053

Table B.23: Normalized Confusion Matrix: mLSTM+sLSTM at 5-second windows ($\pm 2s/\pm 2s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.044 ± 0.036	0.434 ± 0.070	0.068 ± 0.054	0.096 ± 0.026	0.065 ± 0.058	0.293 ± 0.092
Class 1	0.020 ± 0.014	0.406 ± 0.152	0.188 ± 0.074	0.004 ± 0.003	0.093 ± 0.073	0.288 ± 0.152
Class 2	0.059 ± 0.011	0.152 ± 0.049	0.511 ± 0.064	0.046 ± 0.014	0.090 ± 0.042	0.141 ± 0.049
Class 3	0.008 ± 0.011	0.004 ± 0.008	0.019 ± 0.015	0.666 ± 0.107	0.142 ± 0.065	0.161 ± 0.102
Class 4	0.005 ± 0.006	0.018 ± 0.007	0.012 ± 0.009	0.015 ± 0.008	0.552 ± 0.102	0.398 ± 0.092
Class 5	0.030 ± 0.011	0.027 ± 0.007	0.026 ± 0.010	0.006 ± 0.003	0.090 ± 0.029	0.822 ± 0.039

Table B.24: Normalized Confusion Matrix: mLSTM+sLSTM at 7-second windows ($\pm 3s/\pm 3s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.042 ± 0.054	0.393 ± 0.064	0.124 ± 0.133	0.150 ± 0.031	0.016 ± 0.019	0.274 ± 0.071
Class 1	0.012 ± 0.012	0.392 ± 0.052	0.185 ± 0.048	0.008 ± 0.006	0.145 ± 0.066	0.258 ± 0.053
Class 2	0.038 ± 0.032	0.124 ± 0.025	0.562 ± 0.046	0.069 ± 0.016	0.078 ± 0.031	0.129 ± 0.041
Class 3	0.006 ± 0.009	0.004 ± 0.008	0.012 ± 0.010	0.706 ± 0.101	0.094 ± 0.072	0.179 ± 0.079
Class 4	0.004 ± 0.005	0.010 ± 0.004	0.017 ± 0.017	0.023 ± 0.007	0.496 ± 0.123	0.451 ± 0.141
Class 5	0.017 ± 0.016	0.027 ± 0.006	0.024 ± 0.005	0.011 ± 0.004	0.091 ± 0.034	0.829 ± 0.028

Table B.25: Normalized Confusion Matrix: mLSTM+sLSTM at 9-second windows ($\pm 4s/\pm 4s$)

True \ Pred	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5
Class 0	0.017 ± 0.031	0.376 ± 0.063	0.134 ± 0.086	0.128 ± 0.041	0.049 ± 0.069	0.295 ± 0.048
Class 1	0.013 ± 0.009	0.375 ± 0.119	0.194 ± 0.086	0.012 ± 0.011	0.079 ± 0.022	0.327 ± 0.169
Class 2	0.028 ± 0.019	0.103 ± 0.031	0.584 ± 0.091	0.089 ± 0.031	0.080 ± 0.026	0.115 ± 0.061
Class 3	0.002 ± 0.004	0.004 ± 0.008	0.020 ± 0.017	0.729 ± 0.093	0.071 ± 0.035	0.174 ± 0.100
Class 4	0.004 ± 0.003	0.012 ± 0.010	0.009 ± 0.011	0.033 ± 0.017	0.467 ± 0.043	0.474 ± 0.037
Class 5	0.015 ± 0.007	0.025 ± 0.005	0.028 ± 0.018	0.014 ± 0.009	0.085 ± 0.009	0.833 ± 0.029

List of Tables

4.1	Model parameter counts and memory footprint for each architecture variant. All models have comparable complexity, with parameter counts ranging from 2.9M to 3.3M.	32
5.1	Performance comparison across architectures and temporal window configurations. Values are mean $\pm$ std over five seeds using balanced-accuracy checkpointing. Boldface indicates the best value per window.	36
5.2	Linear Transformer vs. sLSTM at 5 s (balanced-accuracy checkpoints). Positive differences favor sLSTM.	38
5.3	Linear Transformer vs. mLSTM at 7 s (balanced-accuracy checkpoints). Positive differences favor mLSTM.	38
5.4	Linear Transformer vs. sLSTM at 9 s (balanced-accuracy checkpoints). Positive differences favor sLSTM.	38
5.5	Training runtime per complete run (100 epochs). Values are mean time with standard deviation in minutes over five seeds.	39
5.6	Normalized confusion matrix for Linear Transformer at 5 s windows, averaged over five seeds. Diagonal entries (bold) show per-class recall; off-diagonal entries show misclassification patterns.	40
5.7	Normalized confusion matrix for sLSTM at 5 s windows, averaged over five seeds.	41
5.8	Normalized confusion matrix for Linear Transformer at 7 s windows, averaged over five seeds.	41
5.9	Normalized confusion matrix for mLSTM at 7 s windows, averaged over five seeds. Best overall performance at this window length.	41
5.10	Normalized confusion matrix for Linear Transformer at 9 s windows, averaged over five seeds.	42
5.11	Normalized confusion matrix for sLSTM at 9 s windows, averaged over five seeds. Best overall balanced accuracy at this window length.	42
5.12	Statistical comparison of per-class recall: Linear Transformer vs. sLSTM at 5 s windows. Tests are paired across seeds; $\alpha = 0.05$	43
5.13	Statistical comparison of per-class recall: Linear Transformer vs. mLSTM at 7 s windows.	43
5.14	Statistical comparison of per-class recall: Linear Transformer vs. sLSTM at 9 s windows.	44
B.1	Individual run metrics for Linear Transformer at 5 s windows.	66
B.2	Individual run metrics for Linear Transformer at 7 s windows.	66
B.3	Individual run metrics for Linear Transformer at 9 s windows.	66
B.4	Individual run metrics for sLSTM at 5 s windows.	67
B.5	Individual run metrics for sLSTM at 7 s windows.	67
B.6	Individual run metrics for sLSTM at 9 s windows.	67
B.7	Individual run metrics for mLSTM at 5 s windows.	67
B.8	Individual run metrics for mLSTM at 7 s windows.	68
B.9	Individual run metrics for mLSTM at 9 s windows.	68

B.10 Individual run metrics for mLSTM+sLSTM at 5 s windows.	68
B.11 Individual run metrics for mLSTM+sLSTM at 7 s windows.	68
B.12 Individual run metrics for mLSTM+sLSTM at 9 s windows.	69
B.13 Complete training runtimes for individual runs (100 epochs each). Times are in hours:minutes format.	69
B.14 Normalized Confusion Matrix: Linear Transformer at 5-second windows ($\pm 2s/\pm 2s$)	70
B.15 Normalized Confusion Matrix: Linear Transformer at 7-second windows ($\pm 3s/\pm 3s$)	70
B.16 Normalized Confusion Matrix: Linear Transformer at 9-second windows ($\pm 4s/\pm 4s$)	71
B.17 Normalized Confusion Matrix: sLSTM at 5-second windows ($\pm 2s/\pm 2s$) .	71
B.18 Normalized Confusion Matrix: sLSTM at 7-second windows ($\pm 3s/\pm 3s$) .	71
B.19 Normalized Confusion Matrix: sLSTM at 9-second windows ($\pm 4s/\pm 4s$) .	71
B.20 Normalized Confusion Matrix: mLSTM at 5-second windows ($\pm 2s/\pm 2s$) .	72
B.21 Normalized Confusion Matrix: mLSTM at 7-second windows ($\pm 3s/\pm 3s$) .	72
B.22 Normalized Confusion Matrix: mLSTM at 9-second windows ($\pm 4s/\pm 4s$) .	72
B.23 Normalized Confusion Matrix: mLSTM+sLSTM at 5-second windows ($\pm 2s/\pm 2s$)	72
B.24 Normalized Confusion Matrix: mLSTM+sLSTM at 7-second windows ($\pm 3s/\pm 3s$)	73
B.25 Normalized Confusion Matrix: mLSTM+sLSTM at 9-second windows ($\pm 4s/\pm 4s$)	73

List of Figures

2.1	BIOT Architecture Overview. Upper: Biosignal tokenization module transforms heterogeneous biosignals into unified "sentences", followed by linear transformer processing. Lower: BIOT's versatility across different learning scenarios including supervised learning, missing data handling, and pre-training approaches. [20]	6
2.2	Biosignal Tokenization Process. Sample 1 demonstrates standard tokenization with four channels (Fp1, Fp2, O1, O2) over 5 seconds. Sample 2 illustrates BIOT's flexibility with mismatched channels (missing O2), variable lengths, and missing values in O1. Different colors represent different channels, and the orange area shows spatial-temporal relationships between tokens. [20]	7
2.3	The Extended LSTM (xLSTM) Family. From left to right: (1) Original LSTM with constant error carousel and gating, (2) New sLSTM and mLSTM cells with exponential gating, (3) xLSTM blocks incorporating residual connections, (4) Complete xLSTM architecture with stacked blocks. [2]	12
2.4	LSTM Limitations Illustrated. Left: Nearest Neighbor Search problem showing LSTM's inability to revise storage decisions (higher MSE indicates worse performance). Right: Rare Token Prediction demonstrating limited storage capacity (higher perplexity indicates worse performance on rare tokens). xLSTM addresses both limitations through exponential gating and matrix memory. [2]	13
3.1	International 10-20 electrode system showing the standardized placement of electrodes on the scalp. The system uses anatomical landmarks (nasion, inion, and preauricular points) to ensure consistent electrode positioning across subjects.	21
3.2	Temporal Central Parasagittal (TCP) montage showing the electrode chains used for bipolar derivations. The longitudinal chains traverse from front to back through temporal regions, while parasagittal chains pass near the midline. This montage is commonly referred to as the "double-banana" configuration.	22
5.1	AUCPR Macro across architectures and temporal window configurations (5s, 7s, 9s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture. At 5s, sLSTM shows a higher median than the Linear Transformer with less overlap.	37
B.1	Balanced accuracy across architectures and temporal window configurations (5s, 7s, 9s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture.	64

- B.2 Cohen's κ across architectures and temporal window configurations (5 s, 7 s, 9 s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture. . 65
- B.3 Weighted F1 across architectures and temporal window configurations (5 s, 7 s, 9 s). Boxplots show the distribution over five random seeds; grouped boxes correspond to window length, and colors denote the architecture. . 65